哈爾濱工業大學（深圳）
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

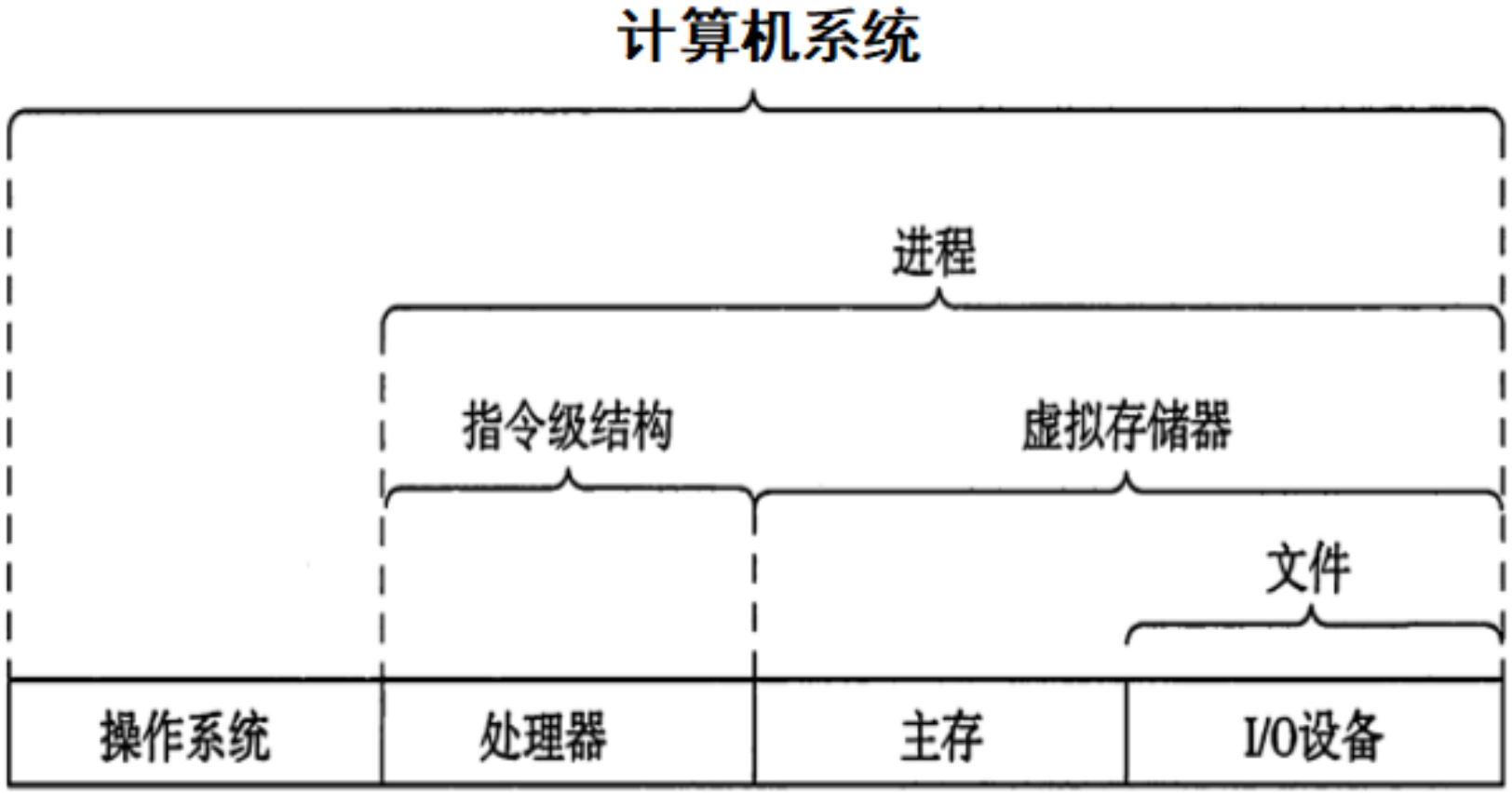
操作系统 (Operating System)

第九章：文件系统

卢瑶 副教授
哈尔滨工业大学（深圳）
2025年秋季

Email：luyao2021@hit.edu.cn

计算机系统的抽象/虚拟化



1、导论与操作系统结构
2、进程与线程
3、并发与同步
4、处理器调度
5、死锁
6、内存管理
7、虚拟内存
8、I/O与存储
9、文件及文件系统

主要内容

- 文件的目录结构
- 文件系统的实现
- EXT2文件系统实现
- Windows的FAT文件系统实现

思考几个小问题

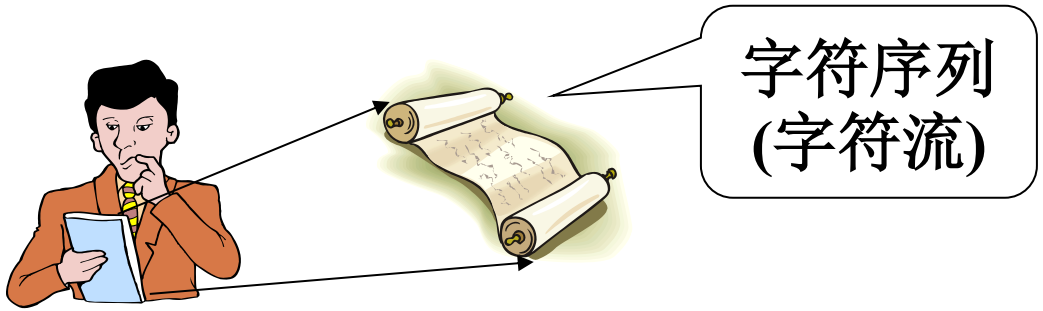
- 拷贝一个1GB的文件 vs 拷贝1M个1KB的小文件？
- 删除一个1GB的文件 vs 删除1M个1KB的小文件？
- 多个小文件总大小和占用磁盘空间大小不一致？
- 同时剪切很多文件，哪种时间快？
  - 1）存放到当前磁盘分区某个文件夹中；
  - 2）存放到其他磁盘分区某个文件夹中
- 针对磁盘的分区或U盘进行格式化是干什么？
- 磁盘碎片清理是干什么？

主要内容

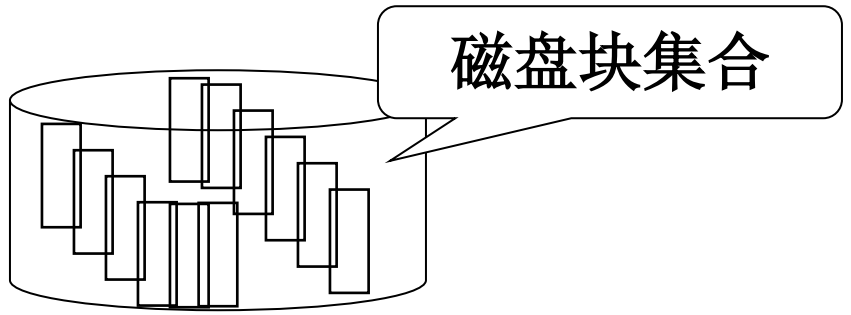
- 文件的目录结构
  - 文件的引入回顾
  - 文件盘块的三种分配方式回顾
  - 文件的目录
- 文件系统的实现
- EXT2文件系统实现
- Windows的FAT文件系统实现

文件: named bytes on disk

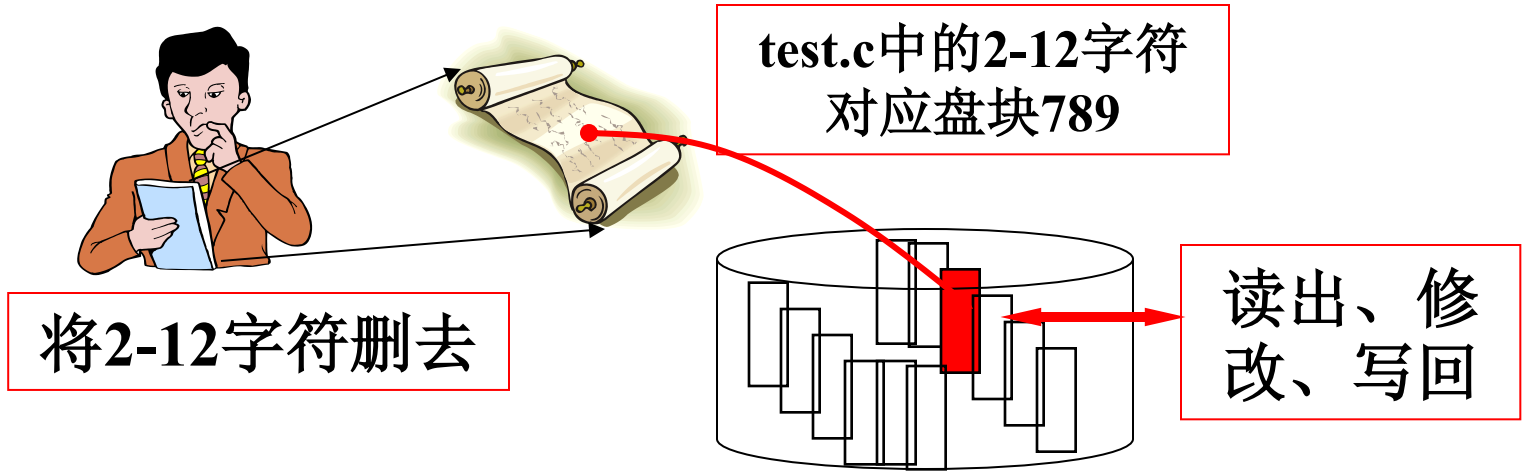
用户眼里的文件



FS眼中的文件



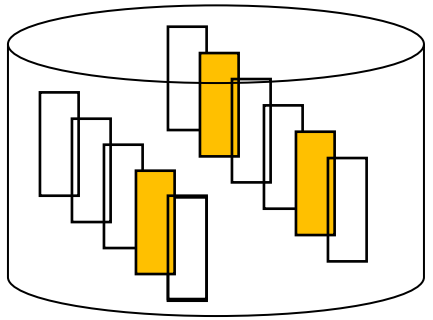
FS的任务: 文件名字和偏移——>磁盘块地址



FS: to have as few disk accesses as possible & have minimal space overhead (group related things)

文件的存取结构

- 文件存取的两种结构：物理结构，逻辑结构



  - 文件的物理结构

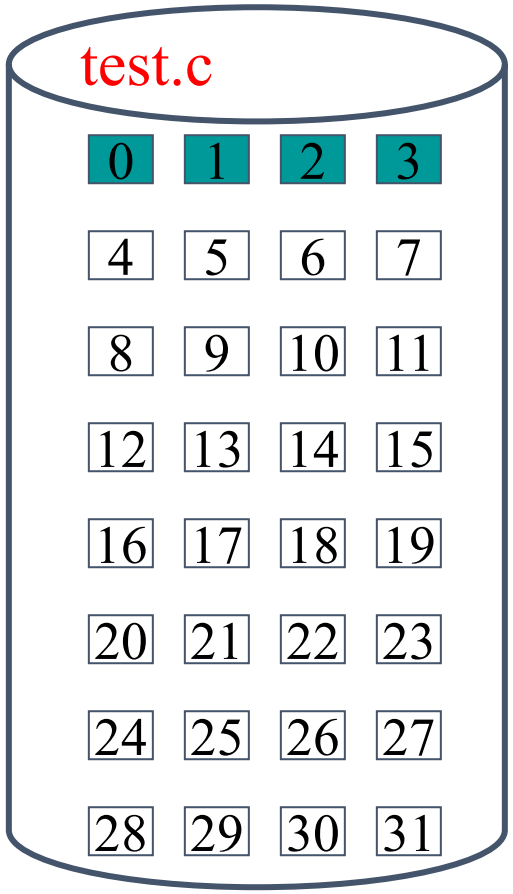
文件的物理结构是信息在物理存储器上的存储方式，常见的有连续存储、链式存储、索引存储。



  - 文件的逻辑结构

用户对信息进行逻辑组织形成的文件结构。为用户提供一种逻辑结构清晰、使用简便的逻辑文件形式。（给用户抽象出来的文件接口）

文件的物理结构：物理盘块连续存储



- 查找文件需要什么信息?

  起始盘块和盘块个数

- 存放在哪里?

  文件描述信息节点中(FCB或File Header)

- 问题：当要存取第2个记录时，应如何操作？

test.c的File Header

文件名	始址	块数
test.c	0	4



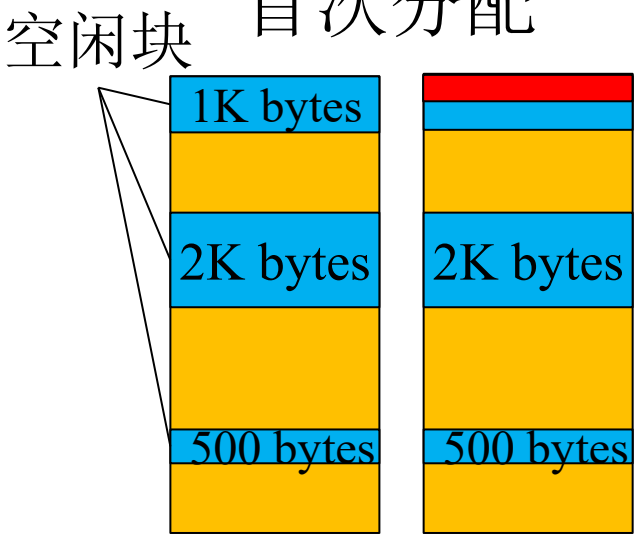
优点：简单、快速

缺点：？ 和连续内存分配相比一下！

物理盘块连续存储vs 内存连续存储

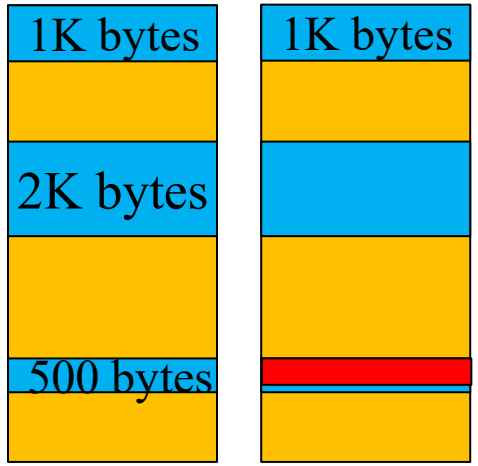
分配400字节

首次分配



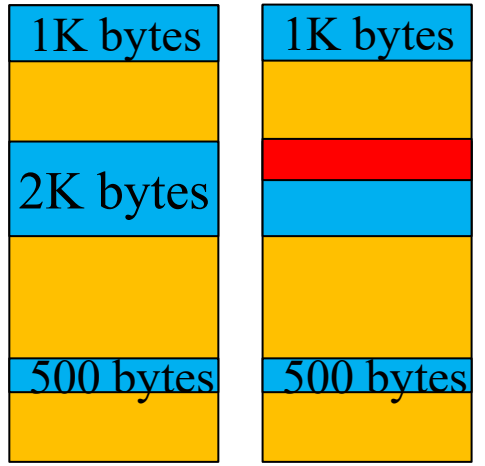
把可用块按地址排序，使用第一个可用空闲块

最佳分配



把可用块按从小到大排序，使用第一个可用空闲块

最差分配

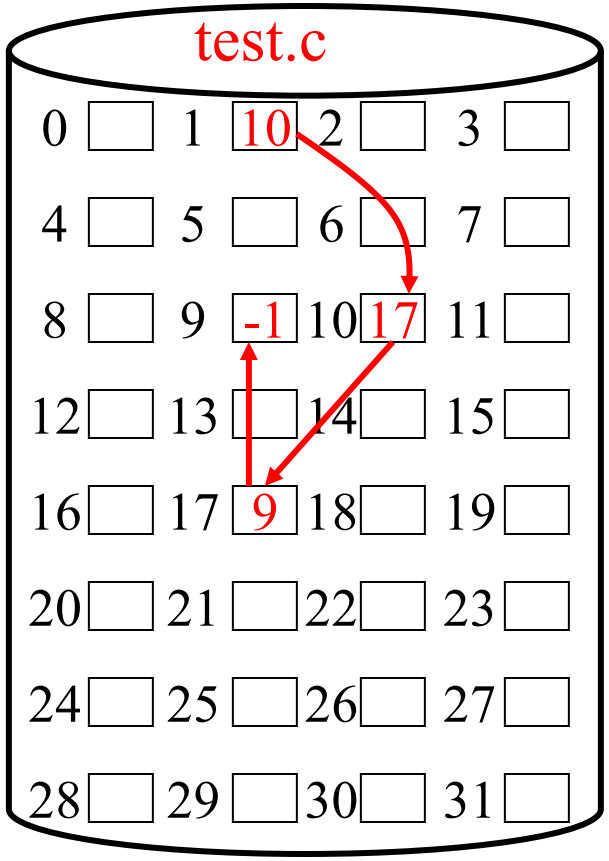


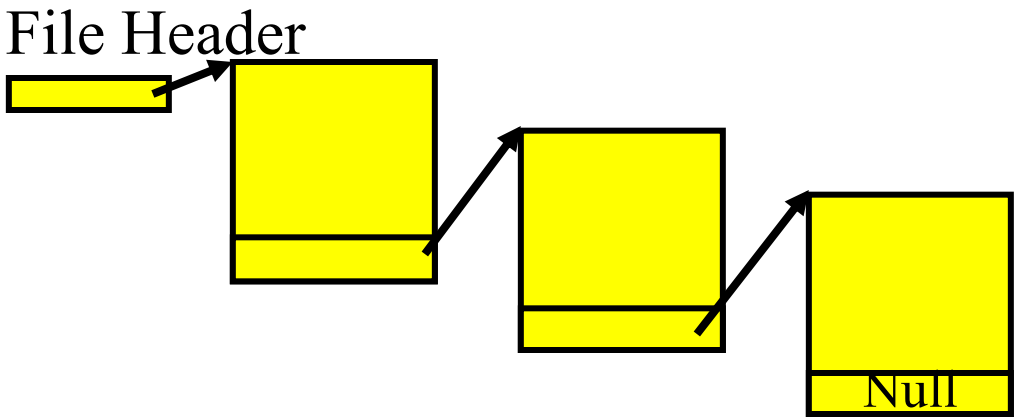
把可用块按从大到小排序，使用第一个可用空闲块



缺点：大量的碎片、不易扩充

文件的物理结构：链式存储





test.c的File Header

文件名	始址
test.c	1

问题：当要存取第 2 个记录时，应如何操作？

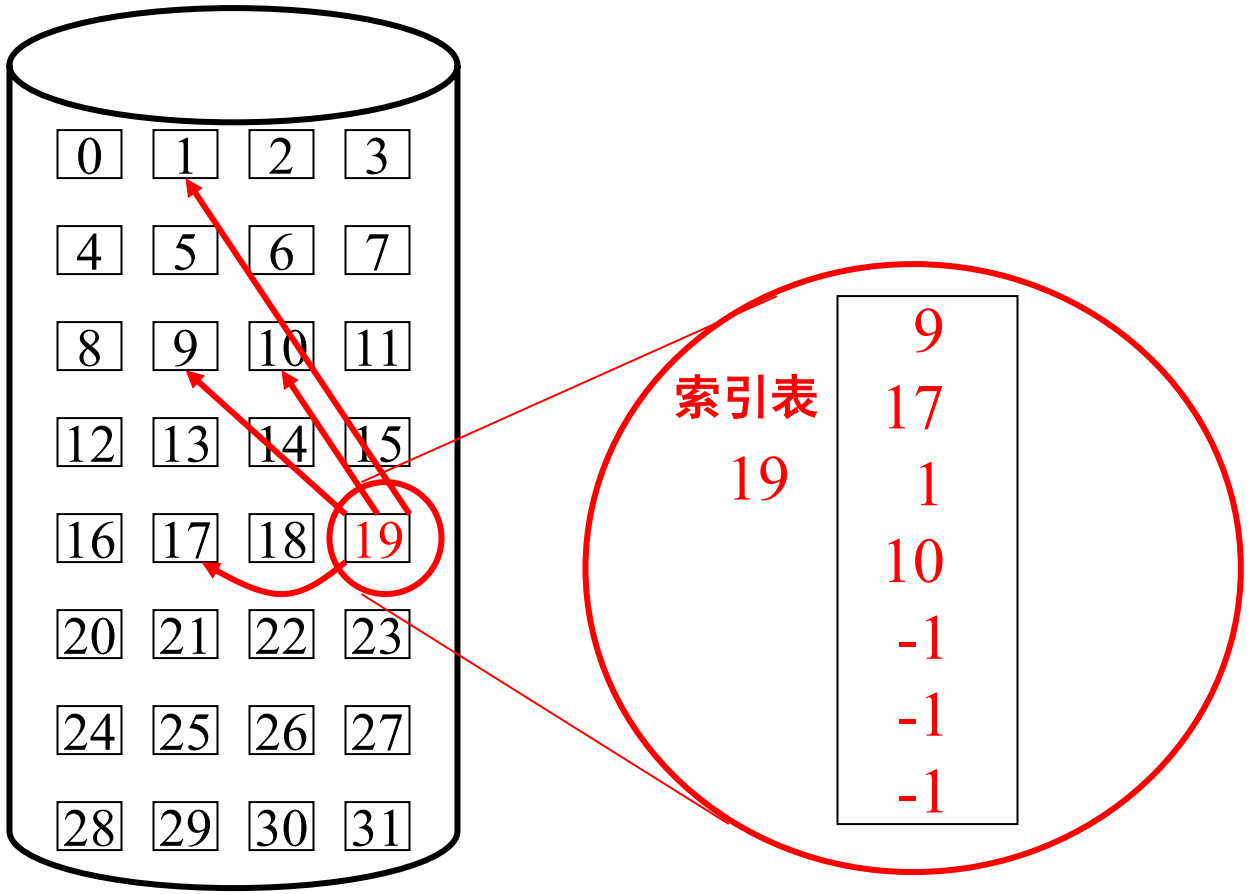




优点：能较好地利用空间，易于对文件进行增生和扩充。

缺点：顺序访问、效率低。

文件的物理结构：索引存储



test.c的File Header

文件名	索引块
test.c	19

问题：当要存取第2 个记录时，应如何操作？



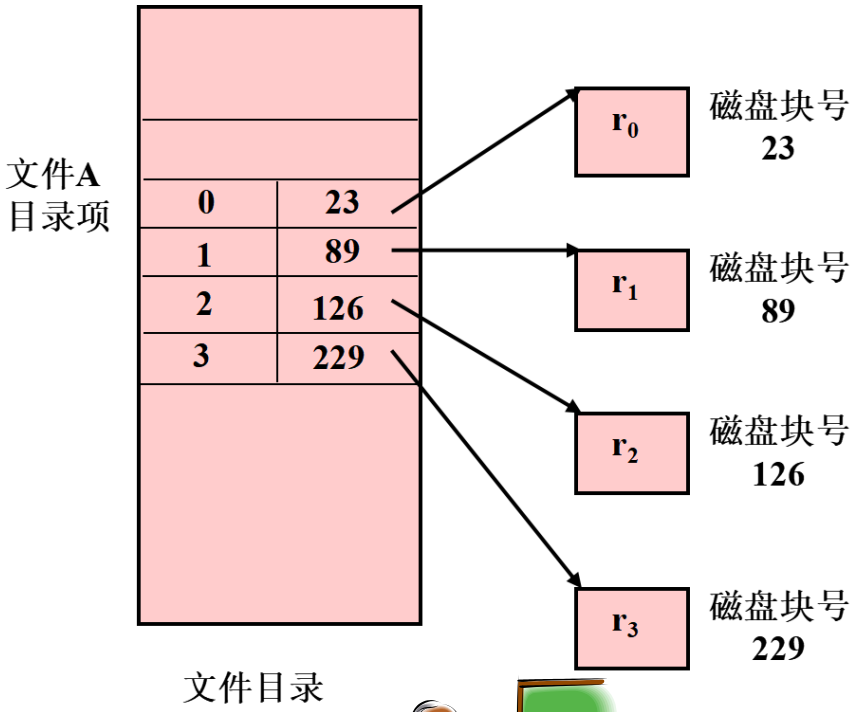


是连续和链式分配的有效折中

文件的物理结构—多级索引

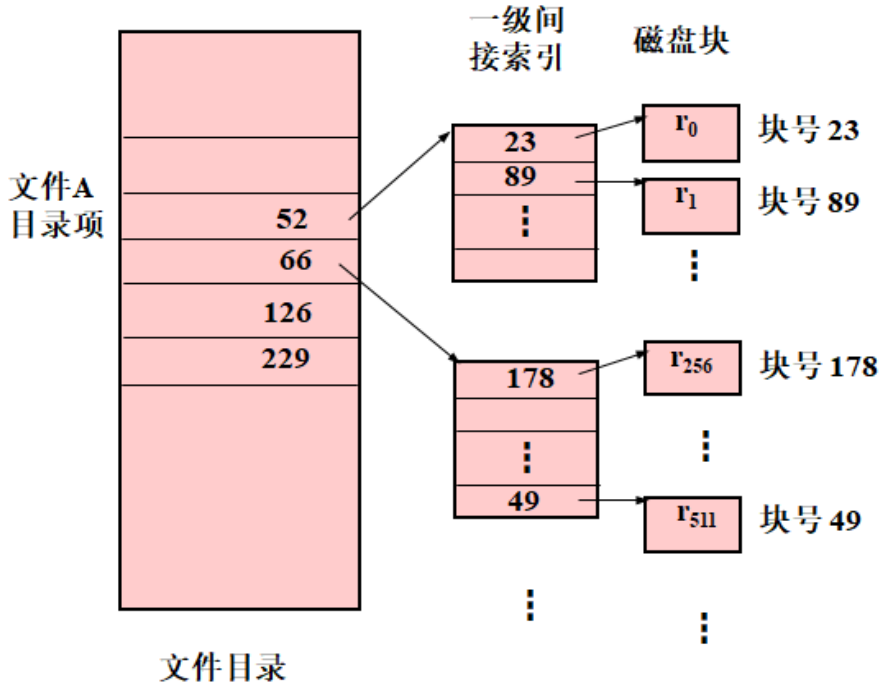
直接索引

- 文件目录项中有一组表项用于索引。每一个表项登记的是逻辑记录所在的磁盘块号。



一级间接索引

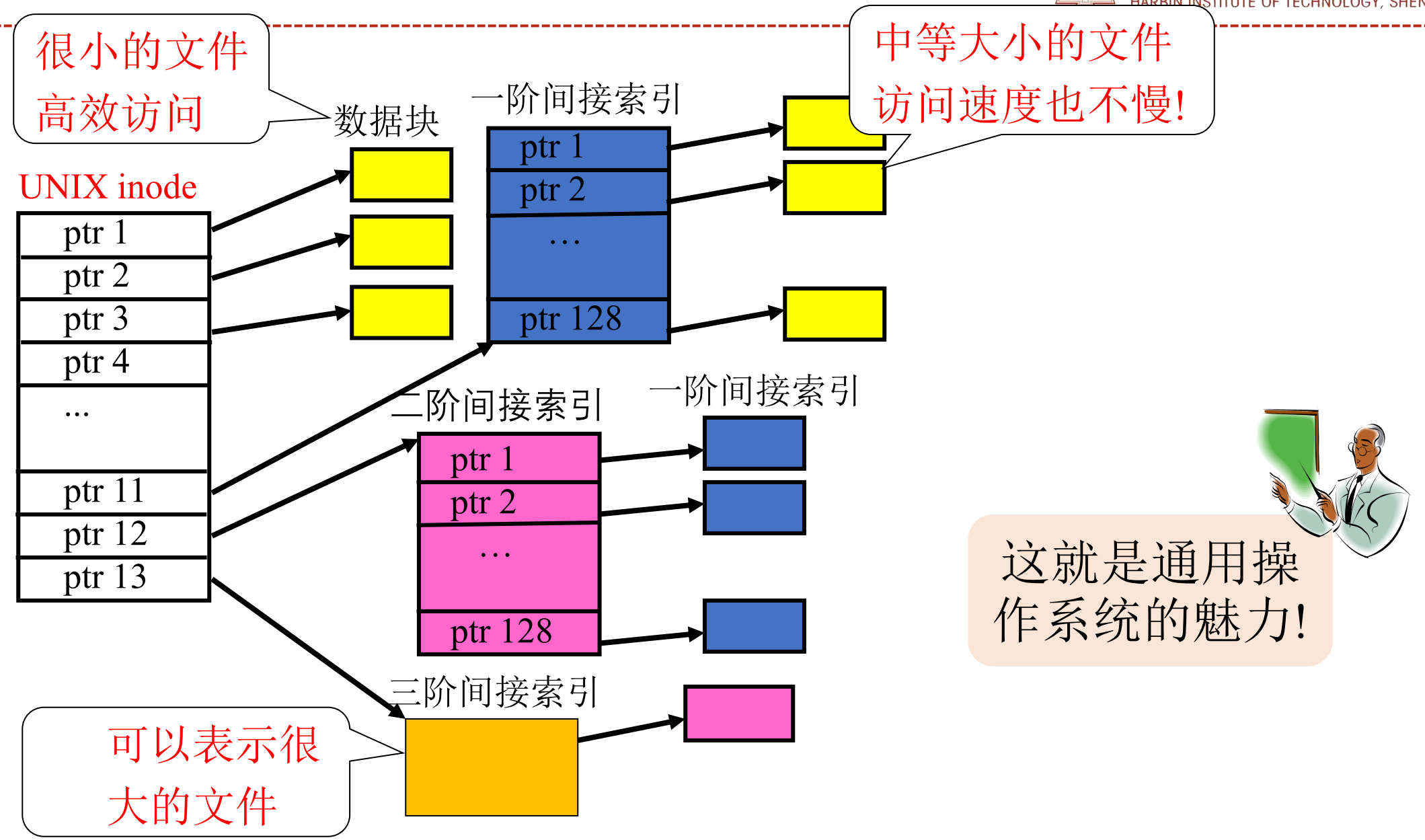
- 文件目录项中有一组表项，其内容登记的是第一级索引表块的块号。第一级索引表块中的索引表项登记的是文件逻辑记录所在的磁盘块号。





类似的，可以设计出二级索引，甚至三级索引。

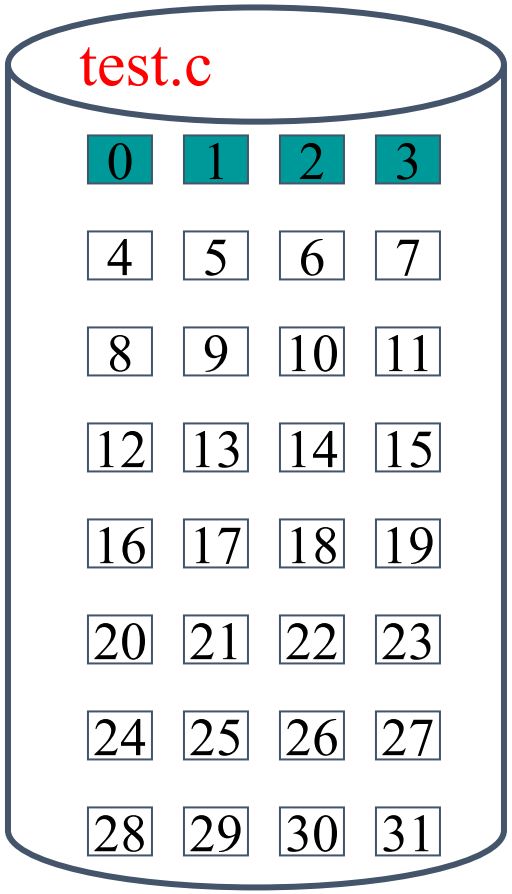
UNIX文件系统的索引结构



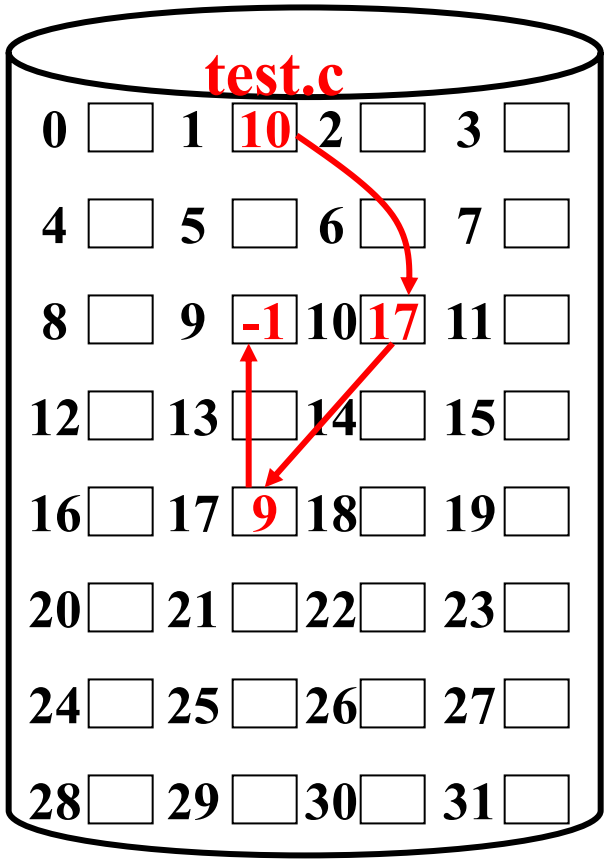
这就是通用操作系统的魅力!

三种基本映射关系

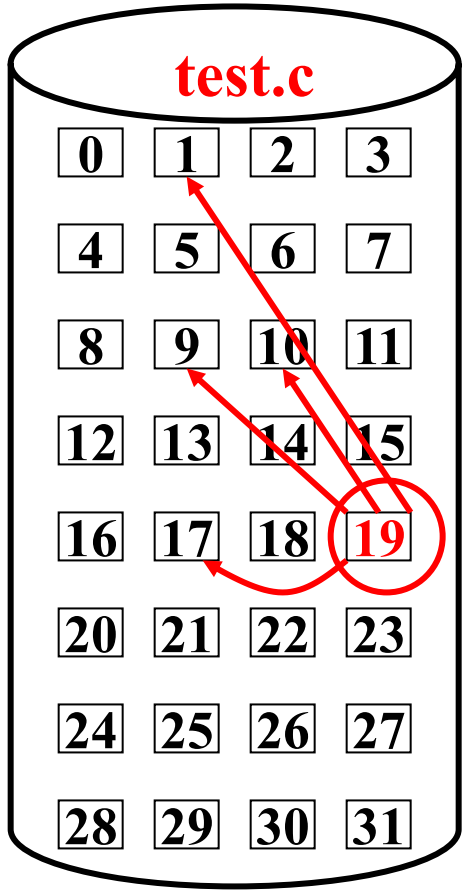
盘块连续分配



盘块链式分配

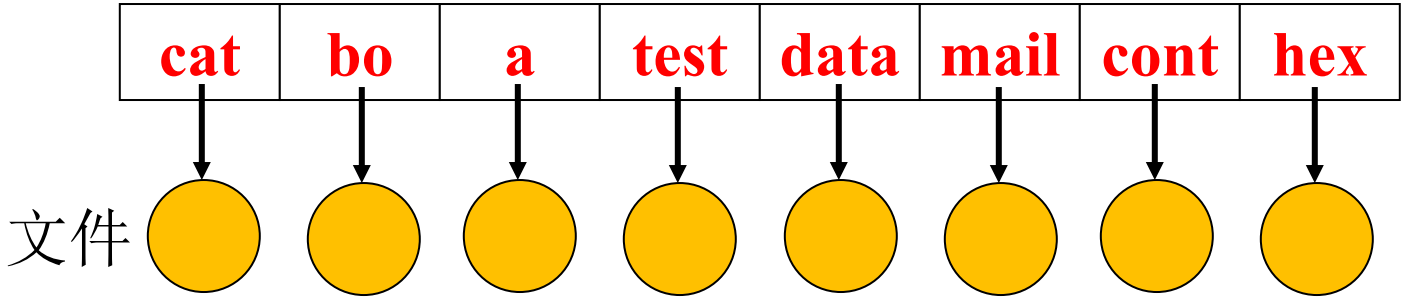


盘块索引分配



文件的逻辑结构

- 文件系统：管理文件的系统。为文件的各类操作和存储（增删改查）提供简单、高效、安全和可靠的管理功能。

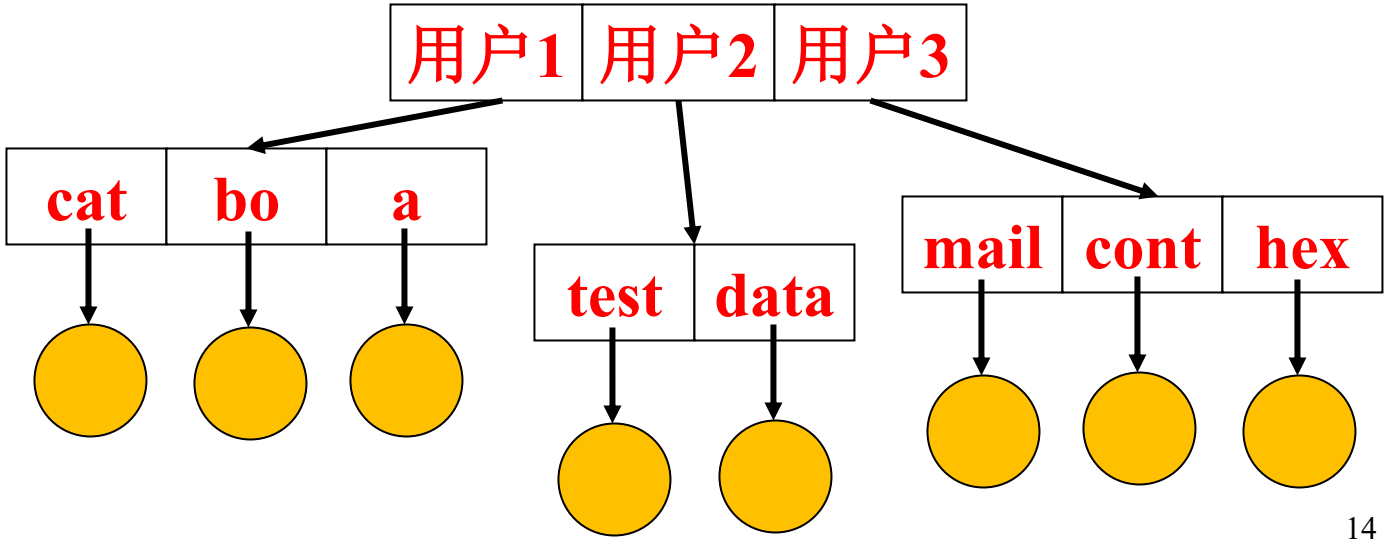


- 所有文件放在一层(一个大集合，例如：桌面)



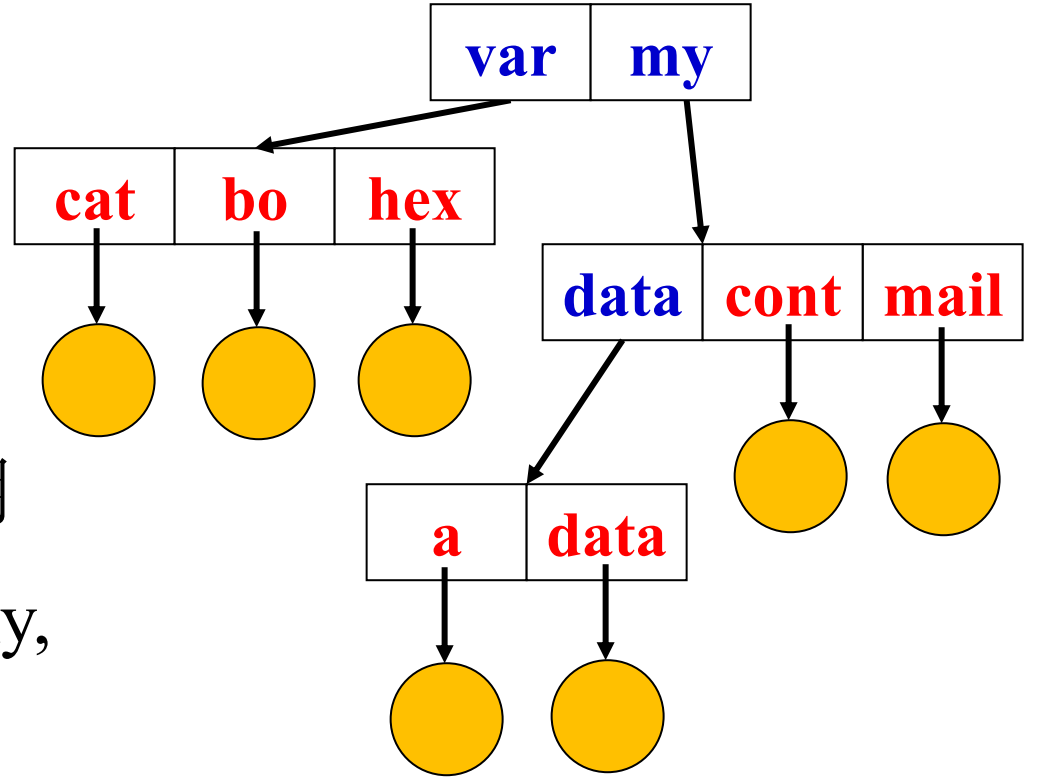
查找困难，不能重名

- 集合划分



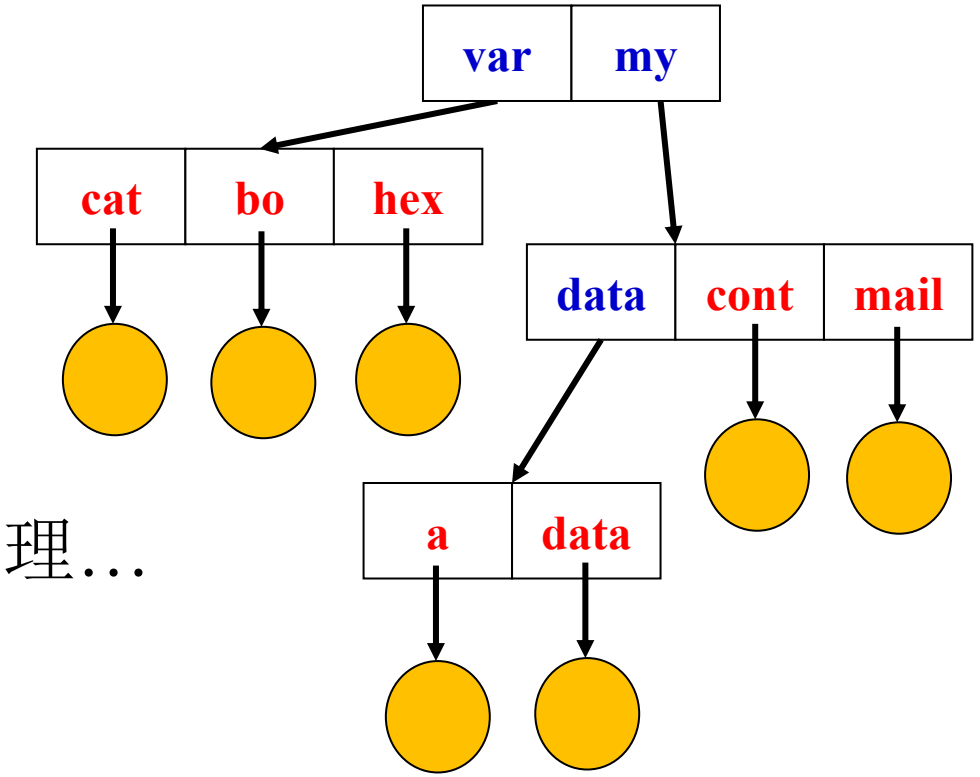
文件的逻辑结构：树状目录

- 将划分后的集合再划分成树状
- 将划分后的集合再进行划分: k次划分后，每个集合中的文件数可以达到为$O(\sqrt[k]{N})$
  - 树状结构扩展性好、表示清晰、最常用
  - 描述“文件的集合”：目录（例如：my, data）
  - 怎么进行“文件集合”的物理表述，即如何在磁盘上存储“目录”？
- 共性：目录要存储信息，文件要存储信息



目录的实现

- 目录是存放“文件的集合”
  - 将文件内容(盘块)放在一起… 显然不行
  - 将文件内容指针(即文件头)放在一起
    - ✓应该是可以的，取决于文件系统如何处理…



- 思考：有了树状目录后会出现什么问题?
  - 出现了路径名：/my/data/a用来定位文件a
  - 路径名⇒ 路径的解析： 输入/my/data/a，获得文件a的文件头，再根据文件头定位文件内容

路径的解析(Name Resolution)

- 输入/my/data/a，获得文件a的文件头
  - 从哪里开始？ 顶层目录(根目录 /)
  - 目录是什么？

```
root@s:~/a# ll
total 8
drwxr-xr-x  2 root root 4096 Jan 30
drwx------ 10 root root 4096 Jan 30
root@s:~/a# stat ./
  File: './'
  Size: 4096        Blocks: 8          IO Block: 4096   directory
Device: fc00h/64512d    Inode: 11534348    Links: 2
Access: (0755/drwxr-xr-x)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2018-01-30 14:16:36.871365855 +0800
Modify: 2018-01-30 14:16:35.563360060 +0800
Change: 2018-01-30 14:16:35.563360060 +0800
```

目标是文件，那么目录的大小是多少？

文件系统中全是文件!

✓应该也是一个文件，文件存放的内容是该目录中所有文件的文件头!

- 解析/my/data/a：

```
res("/my/data/a")
{  fh=FileHeader("/"); //根目录的文件头
   data=ReadData(fh);fh=Find(data,"my");
   data=ReadData(fh);fh=Find(data,"data");
   data=ReadData(fh);fh=Find(data,"a");
   return fh; }
```

放在磁盘确定位置，可在OS初始化时读入!

从路径解析来看目录内容

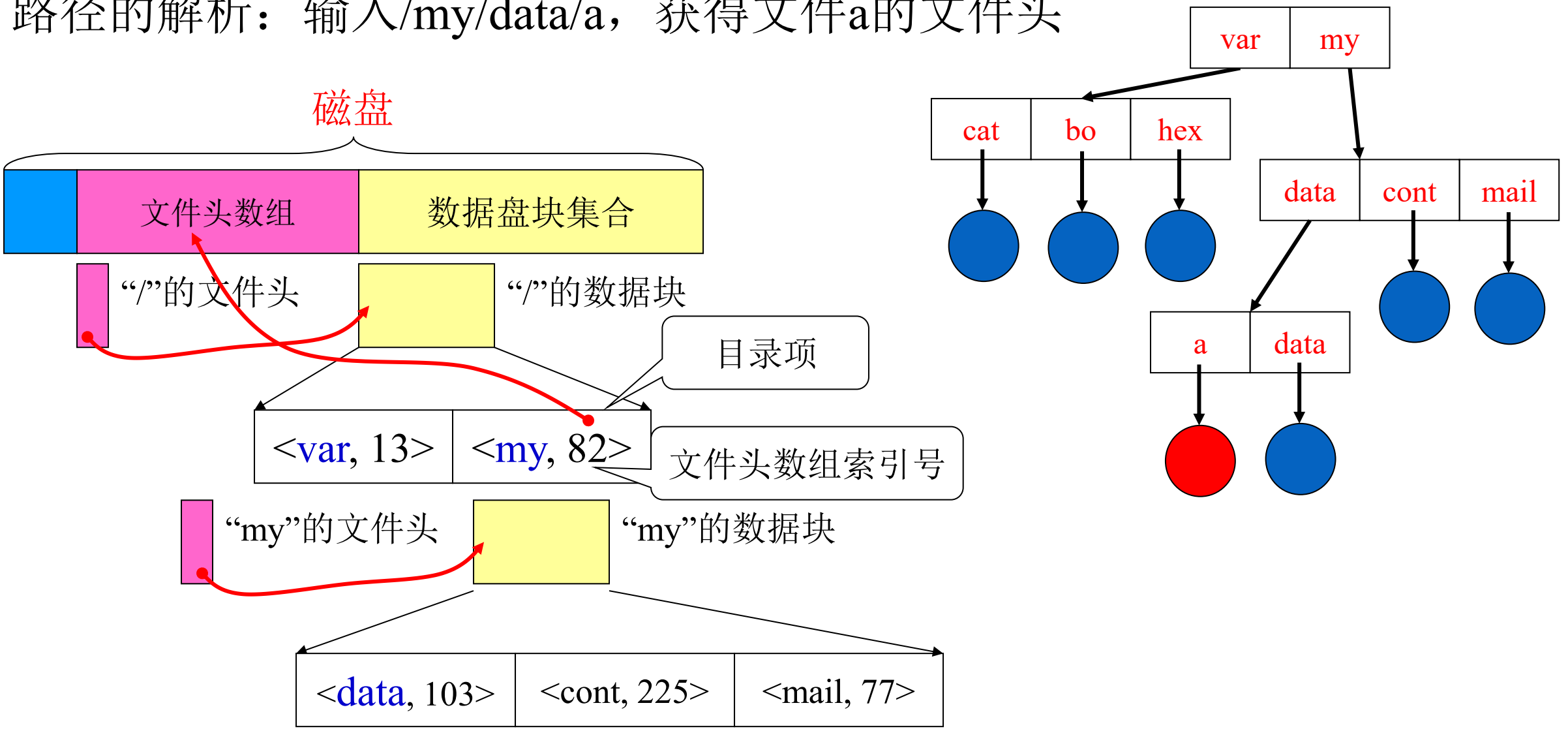
- 显然路径解析的使用频率高，因此效率很重要
  - 如何提高路径解析的效率?
  - 要使语句data=ReadData(fh); fh=Find(**data**,"??"); 效率高，data应该尽可能短!
  - 所以目录文件中不应该存放完整的文件头，可以存放指向文件头的指针
  - 将文件头连续存放(形成了数组)在磁盘的固定位置，文件头指针就是其数组项标号!

文件头尺寸并不小

基址已知、偏移已知就能找到文件头。任何文件的文件头结构相同。

文件的逻辑结构：路径的解析(Name Resolution)

- 路径的解析：输入/my/data/a，获得文件a的文件头



回答这几个小问题

- 拷贝一个1GB的文件 vs 拷贝1M个1KB的小文件？
- 删除一个1GB的文件 vs 删除1M个1KB的小文件？
- 多个小文件总大小和占用磁盘空间大小不一致？
- 同时剪切很多文件，哪种时间快？
  - 1）存放到当前磁盘分区某个文件夹中；
  - 2）存放到其他磁盘分区某个文件夹中
- 针对磁盘的分区或U盘进行格式化是干什么？
- 磁盘碎片清理是干什么？

主要内容

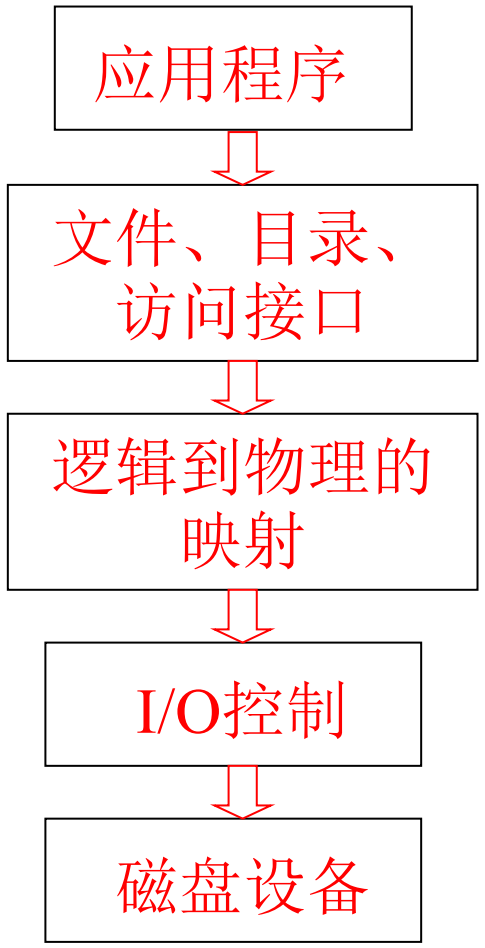
- 文件的目录结构
- 文件系统的实现
  - 文件系统定义
  - 典型的文件系统结构
  - 文件分区空闲块的管理
- EXT2文件系统实现
- Windows的FAT文件系统实现

描述文件系统的实质(定义)

- 文件系统: 将盘块“变”成文件集，方便用户访问

  - 文件系统是“抽象盘块”的一层软件!

就像将CPU资源和地址空间封装成进程一样!



删除test.c 2-12个字节

⇩

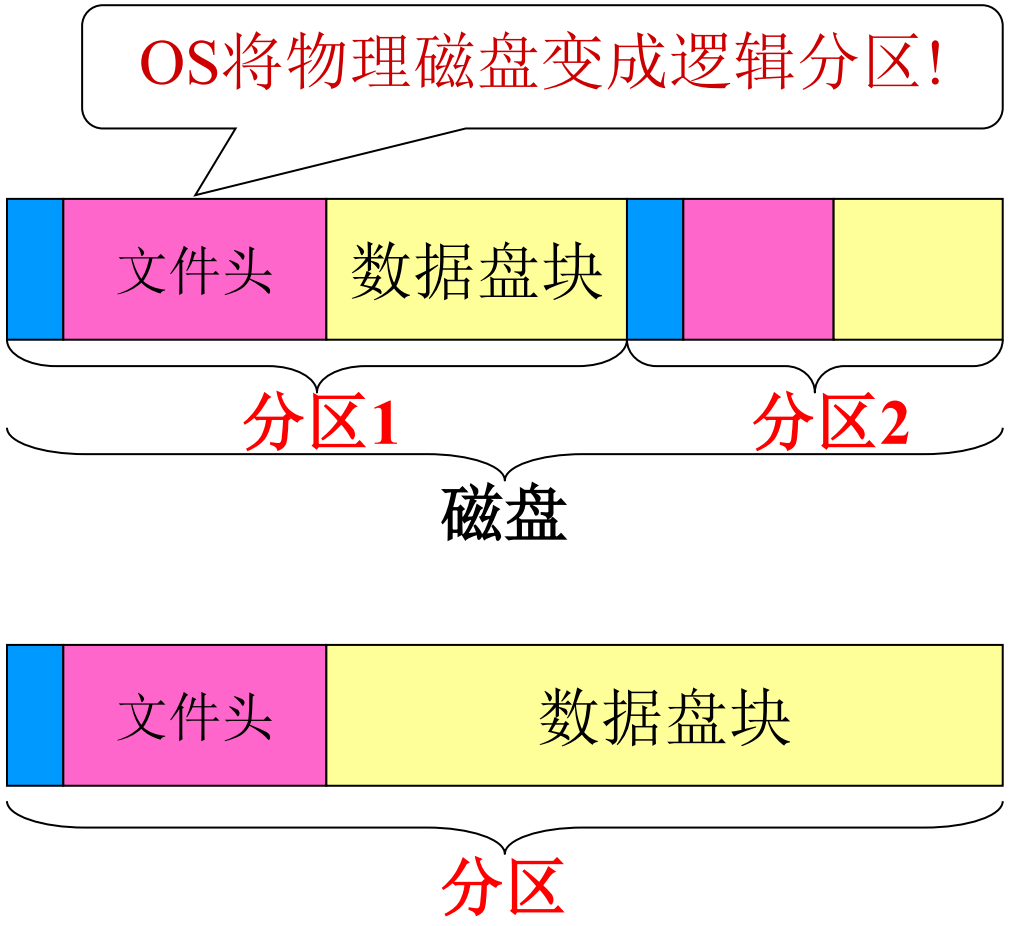
找到test.c的文件头: 需要定位根目录，解析目录…

⇩

根据文件头可以找到2-12字节对应的盘块(789)

⇩

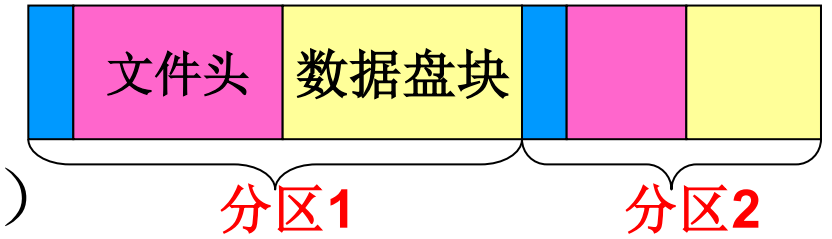
启动DMA，等待789读入，读入后删除2-12字节再启动DMA写出

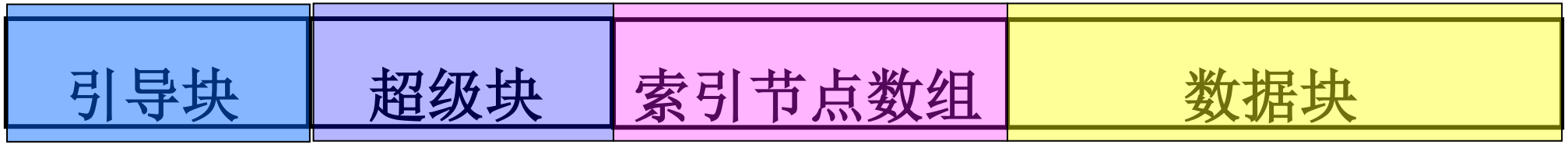


分区的详细结构

■ 典型分区结构：UNIX分区的基本结构

（具体内容在后面的EXT2中介绍）



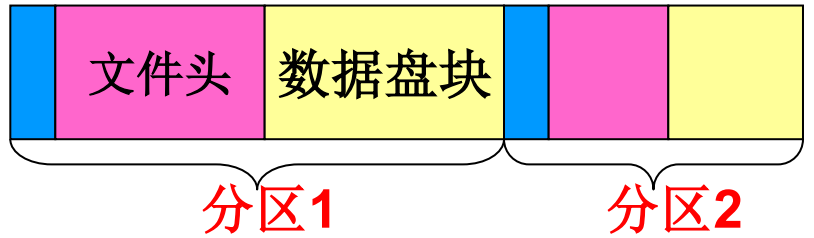


- 引导块存放引导OS的信息，如果该分区中没有OS，则该块为空
- 超级块记录分区基本信息：分区块数；块大小；空闲块数量、指针；空闲文件头数量、指针等。
- 索引节点数组存放所有文件的文件头，UNIX root目录的索引节点号为2
- 数据块，文件内容

```
suryansh@journaldev:~$ sudo tune2fs -l /dev/sda2
tune2fs 1.45.5 (07-Jan-2020)
Filesystem volume name:   <none>
Last mounted on:          /boot
Filesystem UUID:          c784ea6b-c464-4962-8596-5
Filesystem magic number:  0xEF53
Filesystem revision #:    1 (dynamic)
Filesystem features:      has_journal ext_attr resi
 extra_isize metadata_csum
Filesystem flags:         signed_directory_hash
Default mount options:    user_xattr acl
Filesystem state:         clean
Errors behavior:          Continue
Filesystem OS type:       Linux
Inode count:              65536
Block count:              262144
Reserved block count:     13107
Free blocks:              222691
Free inodes:              65224
First block:              0
Block size:               4096
Fragment size:            4096
```

分区空闲盘块的管理

■ 有的盘块被文件使用，空闲盘块如何管理?



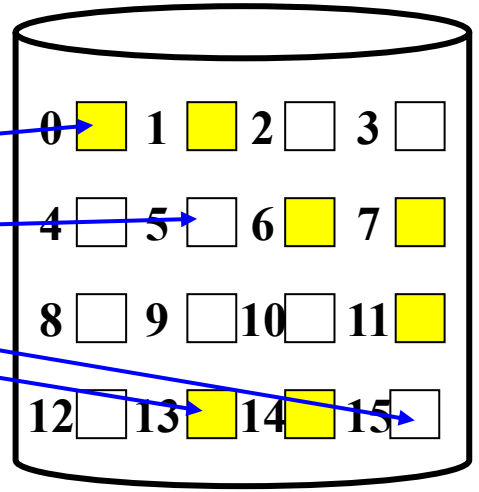
➢组织起来等待文件的使用！ **怎么组织?**

方法1：空闲位图(位向量)…

0011110011101001

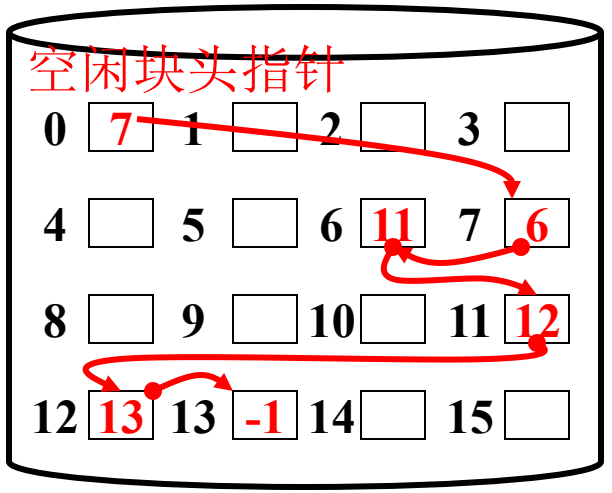
表示磁盘块2,3,4,5,8,9,10,12,15已被占用。

可快速分配连续盘块组，但位向量很大(1G/1k=?)



方法2：空闲链表

分配一个(或少量的)空闲盘块是可以高效工作，但分配多个则慢！



良好运转的文件系统应该高效

- 相比CPU和内存，磁盘读写非常慢!

```
int main(int argc, char* argv[])
{
      int  i , to, *fp, sum = 0;
      to = atoi(argv[1]);
      for(i=1; i<=to; i++)
      {
            sum = sum + i;
            printf(fp,"%d", sum);
       }
}
```

printf语句输出
花费时间

```
C:\>sum 10000000
0.015000 seconds
```

$0.015 * 10^{-7}$ s

用fprintf花费时间

```
C:\>sum 1000
0.859000 seconds
```

$0.859 * 10^{-3}$ s

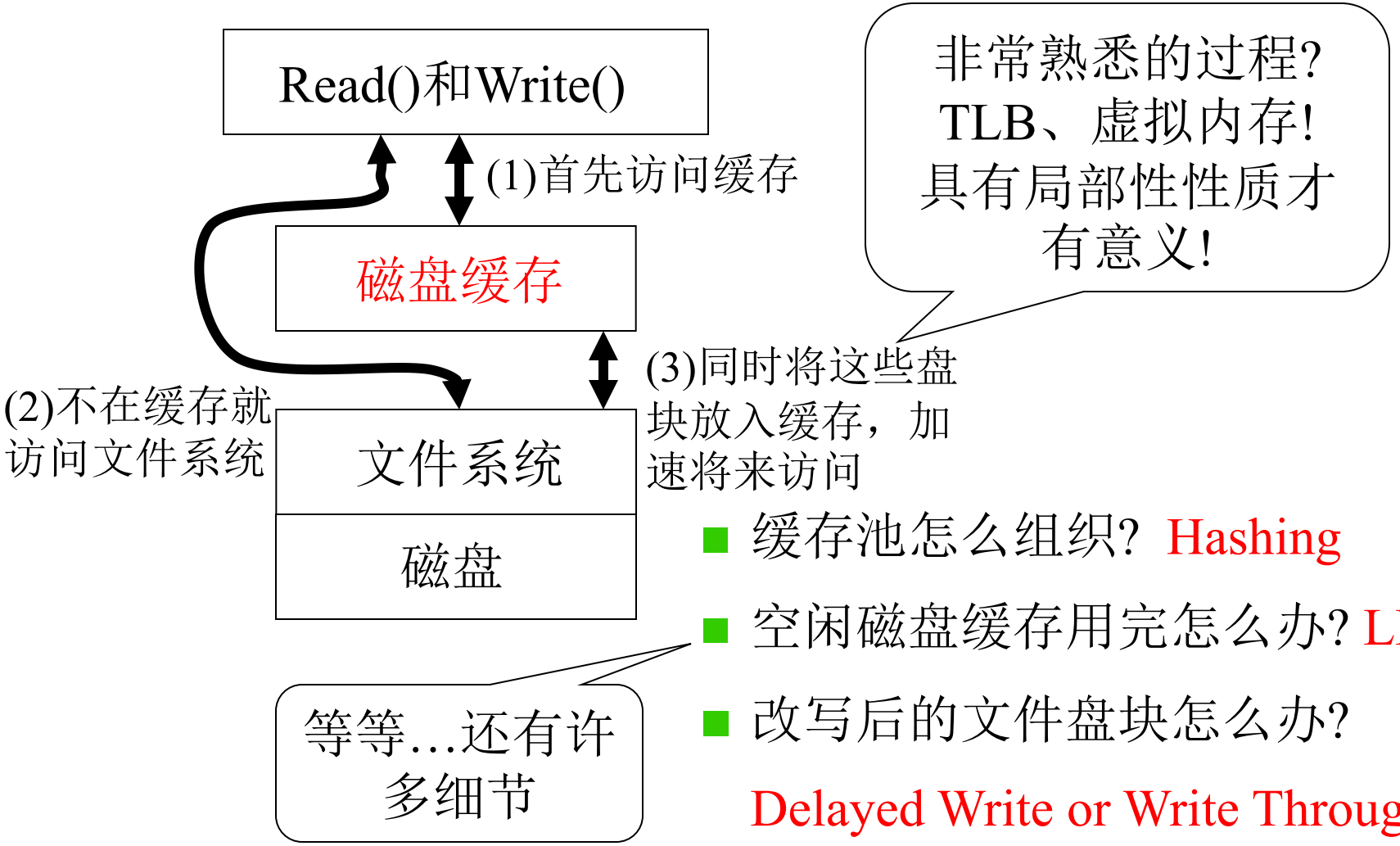
$5.7\times10^5 : 1$

解决速度差异问题的基本手段是?
引入缓存! (因为局部性)



磁盘缓存

- 在内存中缓存磁盘上的部分(很少部分)盘块

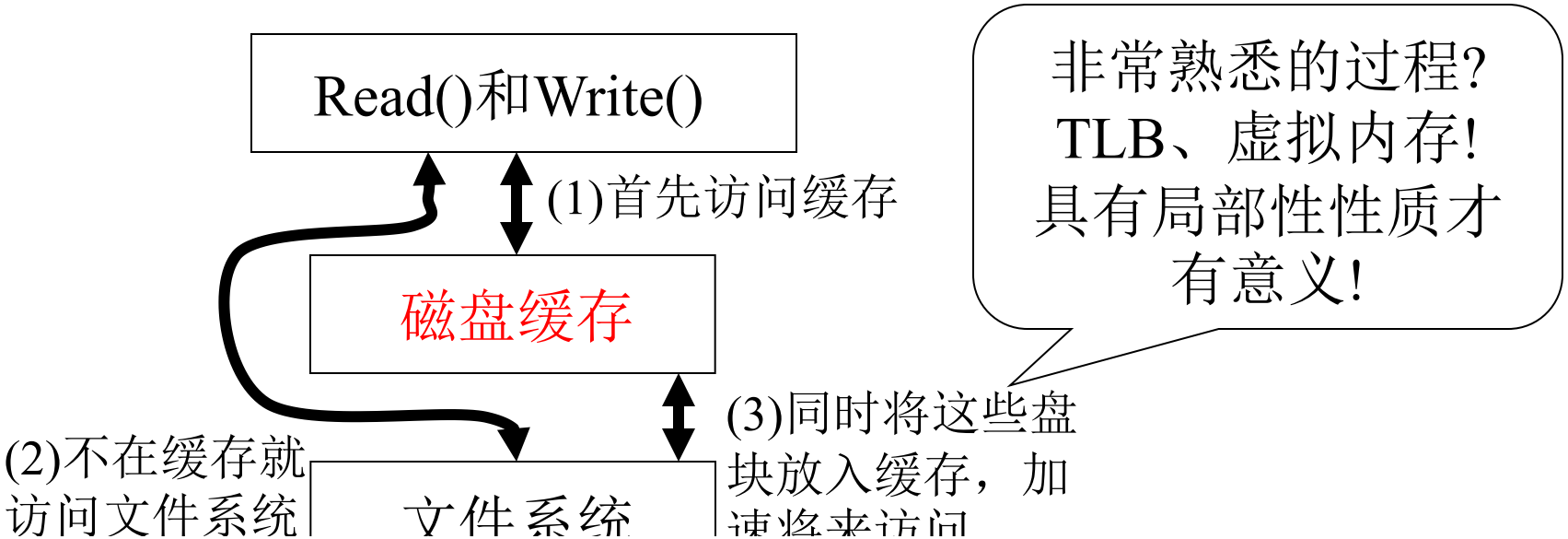


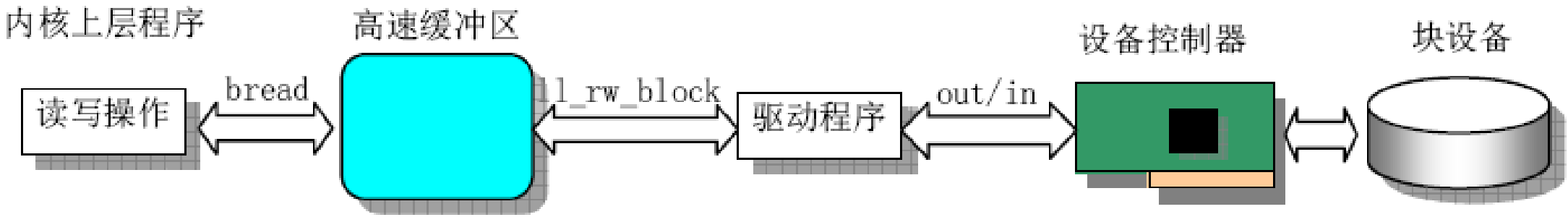
- 缓存池怎么组织? Hashing
- 空闲磁盘缓存用完怎么办? LRU
- 改写后的文件盘块怎么办?

  Delayed Write or Write Through

磁盘缓存

- 在内存中缓存磁盘上的部分(很少部分)盘块





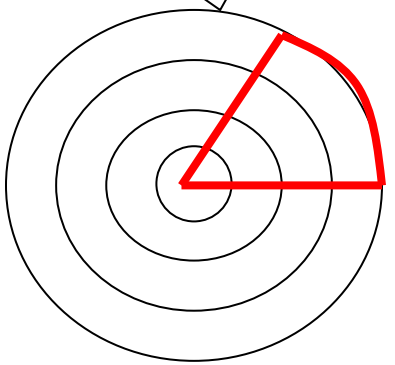
Delayed Write or Write Through

其他的提高文件访问效率的技术…

- 某些目录文件的FCB可以常驻内存
  - 根目录 / 的FCB常驻内存，因为许多目录解析都从此处开始
  - 当前目录(pwd)，如gcc 1.c。其FCB可驻内存?
- ls -l的使用非常频繁
  - 怎么才能快速执行?
  - 同一目录中的文件inode在一个柱面(组)!
  - 需要重新设计inode、目录文件分配算法…
- 显然，还有许多提高文件系统效率的技术……
  - **需要自己去整理，这是操作系统课程显著特点之一**

一段时间大多数文件访问集中在一个目录中(**局部性**)

每个柱面组都有inode，空闲盘块…



良好运转的文件系统应该提供保护

- 文件用来存放用户的信息，用户应该能控制对文件的访问：如只允许读
  - 文件关联权限，哪些权限？放在哪里？
    - ✓**权限：**读/写/执行(r/w/x)，当然是放在文件头中！
    - ✓一实例：**drwxrwxrwx** root staff test/

不同用户具有不同权限 | owner id | group id

  - 何时设置或检查权限？
    - ✓访问文件是由进程发起的，进程是由用户启动的：(1)PCB中有uid和gid; (2)fork时设置; (3)这些信息在登录时收集(从tty); (4)文件访问时校验

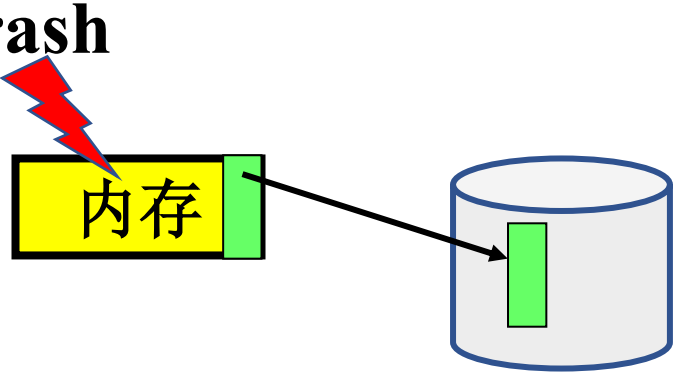
又将许多东西联系在了一起…OS魅力所在！

回忆:进程控制块PCB

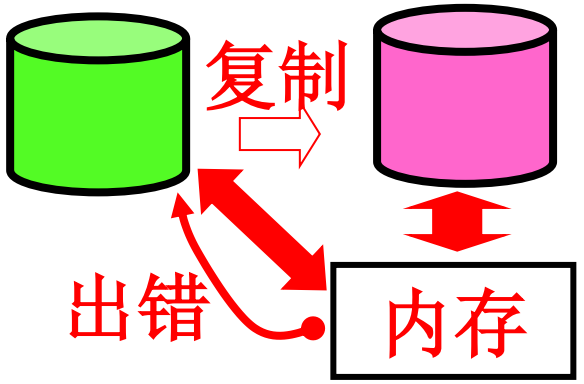

```
struct task_struct {
/*-----------------------------------------------*/
    long state;    // 进程运行状态（-1 不可运行，0 可运行，>0 以停止）
    long counter; // 任务运行时间片，递减到 0 是说明时间片用完
    long priority;     // 任务运行优先数，刚开始是 counter＝priority
    long signal;  // 任务的信号位图，信号值=偏移+1
    struct sigaction sigaction[32];  //信号执行属性结构，对应信号将要执行的操作和标志信息
    long blocked; // 信号屏蔽码
/*--------------- various fields---------------------- */
    int exit_code;     // 任务退出码，当任务结束时其父进程会读取
    unsigned long start_code, end_code, end_data, brk, start_stack;
        // start_code      代码段起始的线性地址
        // end_code    代码段长度
        // end_data    代码段长度+数据段长度
        // brk         代码段长度+数据段长度+bss 段长度
        // start_stack     堆栈段起始线性地址
    long pid, father, pgrp, session, leader;
        // pid 进程号，father 父进程号，pgrp 父进程组号，session 会话号，leader 会话首领
    unsigned short uid, euid, suid;
        // uid 用户标 id，euid 有效用户 id，suid 保存的用户 id
    unsigned short gid, egid, sgid;
        // gid 组 id，egid 有效组 id，sgid 保存组 id
    long alarm;    // 报警定时值
```

良好运转的文件系统似乎也应该容错

- 有过这样的经历?
  - 早晨起来发现写了数月论文打不开了(如因下一块的link断了)…

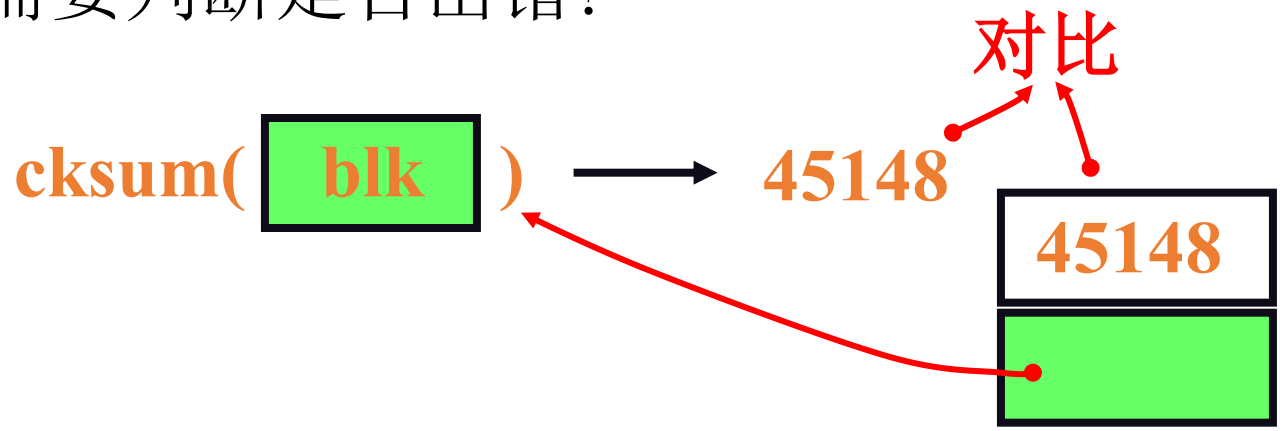


- 错误是难免的：误操作、突然断电、无处不在的电磁干扰… **怎么办？错误避免还是错误恢复…**
- RAID(Redundant Arrays of Independent Disks)
- 基本思想就是冗余(R)：如在镜像磁盘上备份数据，发现错误时拷贝镜像磁盘(恢复)

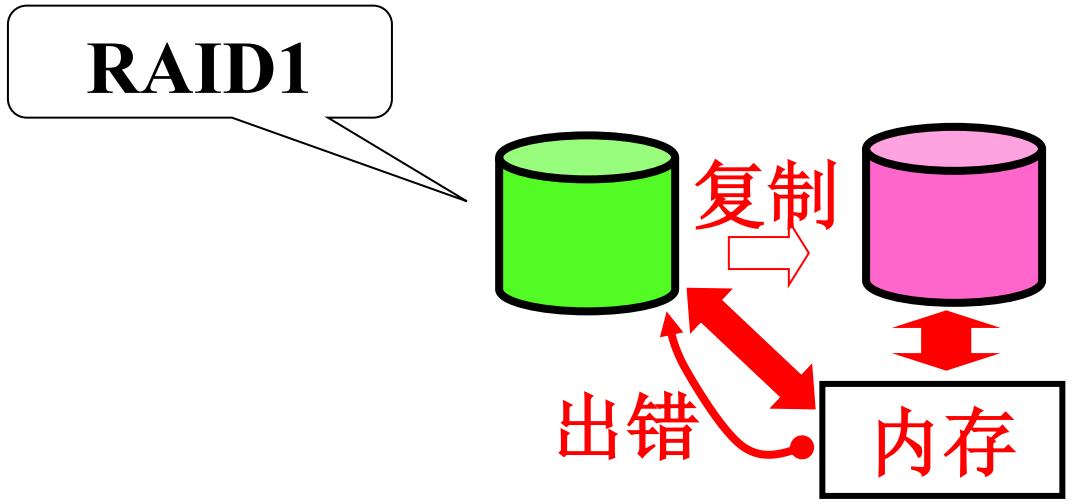


RAID的简单实现

- 首先需要判断是否出错?



- 此时的磁盘读写
  - (1)每次写两个磁盘都写
  - (2)从磁盘A读，发现错误转向B
  - (3)A有错时将B的数据拷到A
- 磁盘利用率50%



- 校验数据分布在多个盘上，磁盘互相恢复数据

避免校验块的更新，全部拥塞到某个磁盘上

磁盘条

异或运算

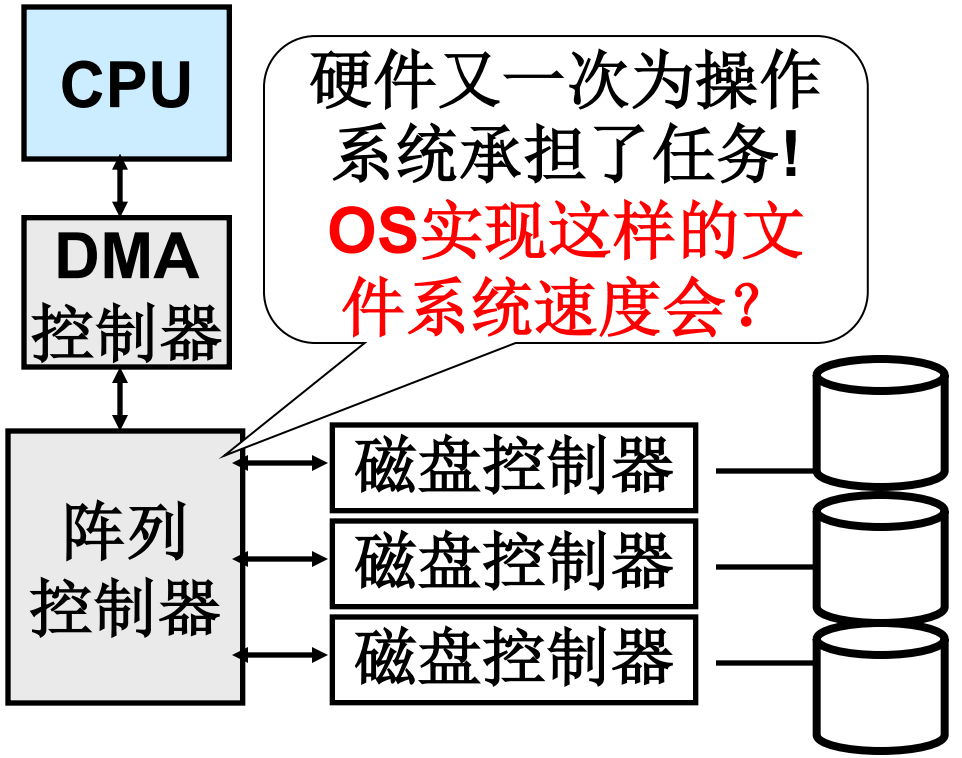
$P0=D0⊕D1⊕D2⊕D3$

$D3=D0⊕D1⊕D2⊕P0$

磁盘阵列统一编址

D0	D1	D2	D3	P0
D4	D5	D6	P1	D7
D8	D9	P2	D10	D11
D12	P3	D13	D14	D15
P4	D16	D17	D18	D19
D20	D21	D22	D23	P5

磁盘1 磁盘2 磁盘3 磁盘4 磁盘5



拓展阅读

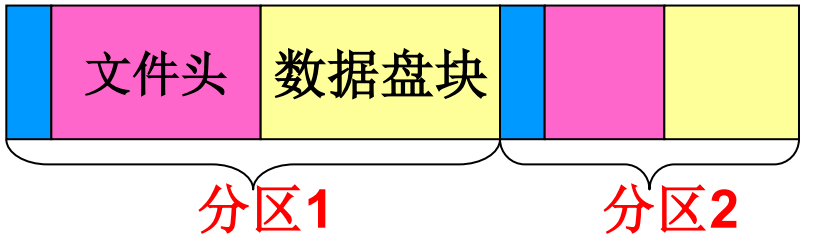
- PRO: A popularity-based multi-threaded reconstruction optimization for RAID-structured storage systems @ FAST 2007
- Workout: I/O workload outsourcing for boosting RAID reconstruction performance @ FAST 2009

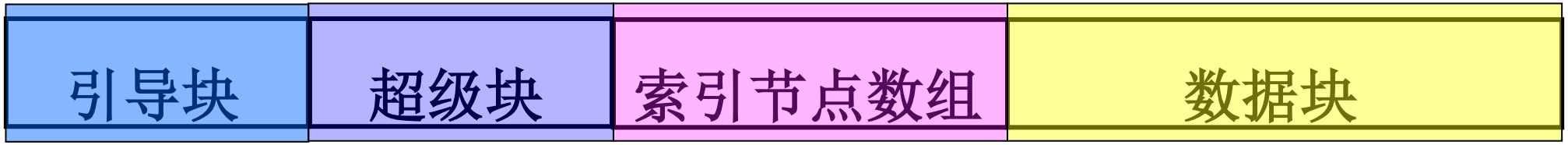
主要内容

- 文件的目录结构
- 文件系统的实现
- EXT2文件系统实现（实验五）
- Windows的FAT文件系统实现

回顾：分区的详细结构

■ 典型分区结构：UNIX分区的基本结构

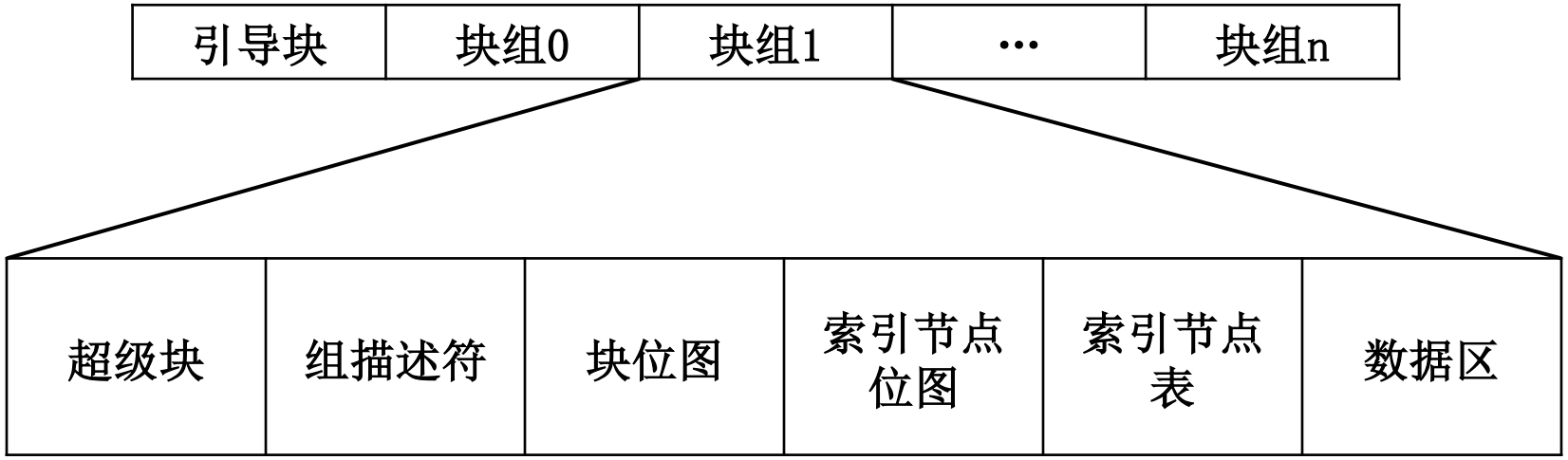




- 引导块存放引导OS的信息，如果该分区中没有OS，则该块为空
- 超级块记录分区基本信息：分区块数；块大小；空闲块数量、指针；空闲文件头数量、指针等
- 索引节点数组存放所有文件的文件头，UNIX root目录的索引节点号为2
- 数据块，文件内容

EXT2文件系统

- EXT2文件系统把逻辑分区划分为块组，并且从0开始编号。每个块组包含的等量的物理块(即块组大小是相同的；物理分区最后一个块组可能小些)；在块组的数据块中存储文件或目录；

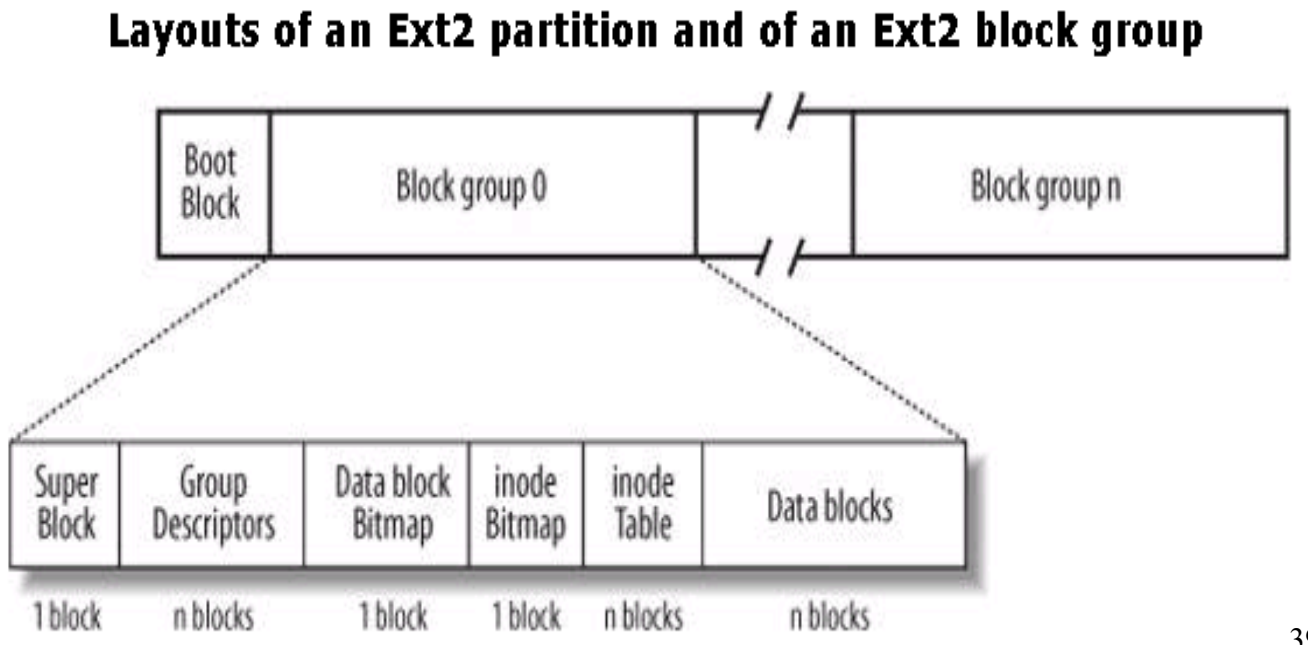


- 上图中启动块/引导块（Boot Block）的大小是确定的，用来存储磁盘分区信息和启动信息，任何文件系统都不能使用启动块。启动块之后才是ext2文件系统的开始。

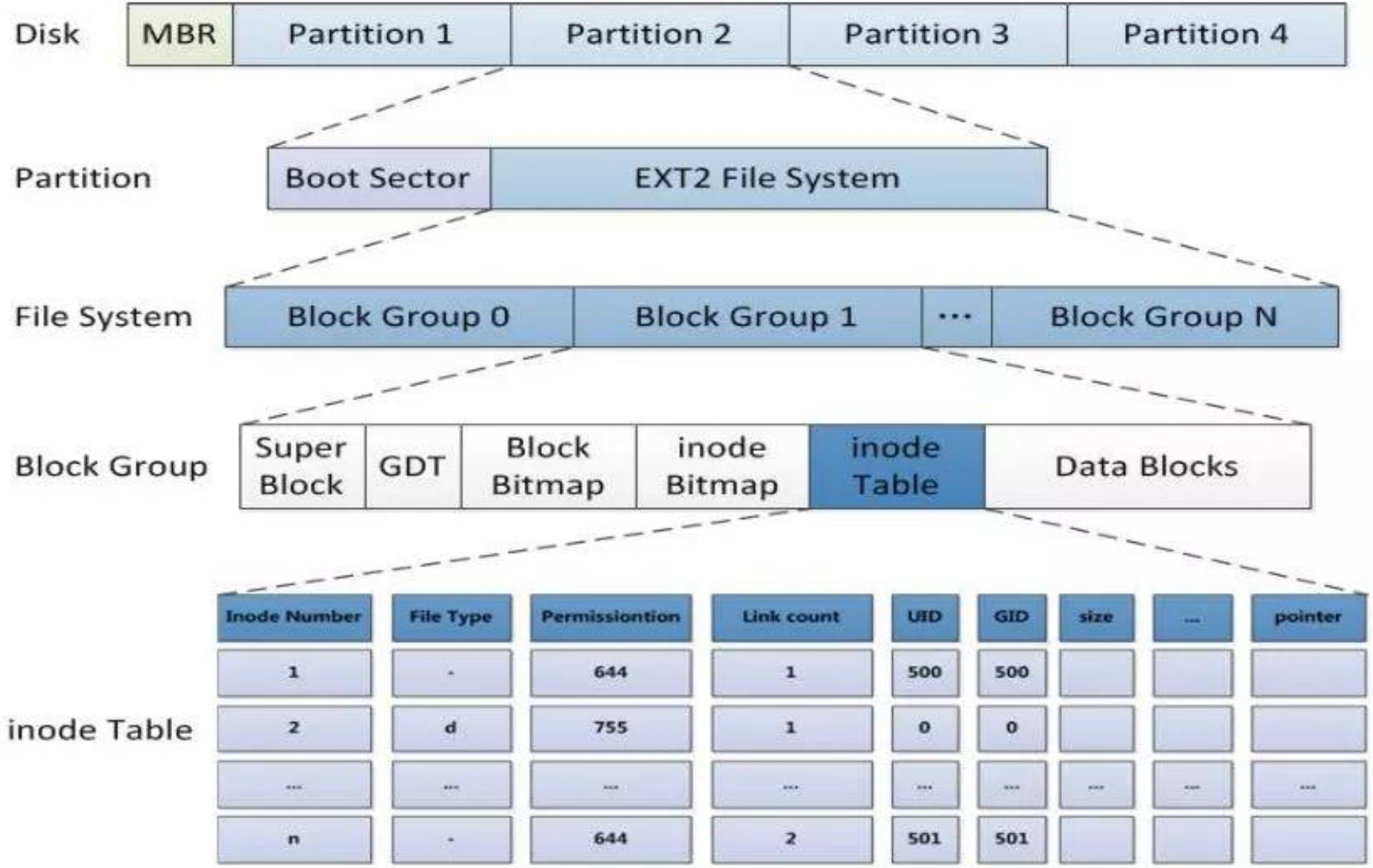
EXT2文件系统的磁盘组织

- 除了引导扇区之外，EXT2磁盘分区被顺序划分为若干个磁盘块组（Block Group）。每个块组按照相同的方式组织，具有相同的大小。

- EXT2磁盘块组中的磁盘块按顺序被组织成：
  - 一个用作超级块的磁盘块：存放了文件系统超级块的一个拷贝；
  - 多个记录组描述符的磁盘块；
  - 1个记录数据块位图的磁盘块；
  - 1个记录索引结点位图的磁盘块；
  - 多个用作索引结点表的磁盘块；
  - 多个用作数据块的磁盘块。



Ext2全貌图


Disk
MBR
Partition 1
Partition 2
Partition 3
Partition 4
Partition
Boot Sector
EXT2 File System
File System
Block Group 0
Block Group 1
···
Block Group N
Block Group
Super Block
GDT
Block Bitmap
inode Bitmap
inode Table
Data Blocks
inode Table
Inode Number
File Type
Permissiontion
Link count
UID
GID
size
...
pointer
1
-
644
1
500
500
2
d
755
1
0
0
...
...
...
...
...
...
...
...
...
n
-
644
2
501
501

EXT2的超级块

- 描述整个分区的文件系统信息，如块大小、版本号、上次mount时间等。
- 每个块组的第一个磁盘块用来保存所在EXT2 fs的超级块
- 多个块组中的超级块形成冗余
  - 在某个或少数几个超级块被破坏时，可用于恢复被破坏的超级块信息。
  - 系统运行期间，把超级块复制到系统缓冲区内，只需把块组0的超级块读入内存，其它块组的超级块做为备份

```
suryansh@journaldev:~$ sudo tune2fs -l /dev/sda2
tune2fs 1.45.5 (07-Jan-2020)
Filesystem volume name:   <none>
Last mounted on:          /boot
Filesystem UUID:          c784ea6b-c464-4962-8596-5
Filesystem magic number:  0xEF53
Filesystem revision #:    1 (dynamic)
Filesystem features:      has_journal ext_attr resi
 extra_isize metadata_csum
Filesystem flags:         signed_directory_hash
Default mount options:    user_xattr acl
Filesystem state:         clean
Errors behavior:          Continue
Filesystem OS type:       Linux
Inode count:              65536
Block count:              262144
Reserved block count:     13107
Free blocks:              222691
Free inodes:              65224
First block:              0
Block size:               4096
Fragment size:            4096
```

超级块数据结构

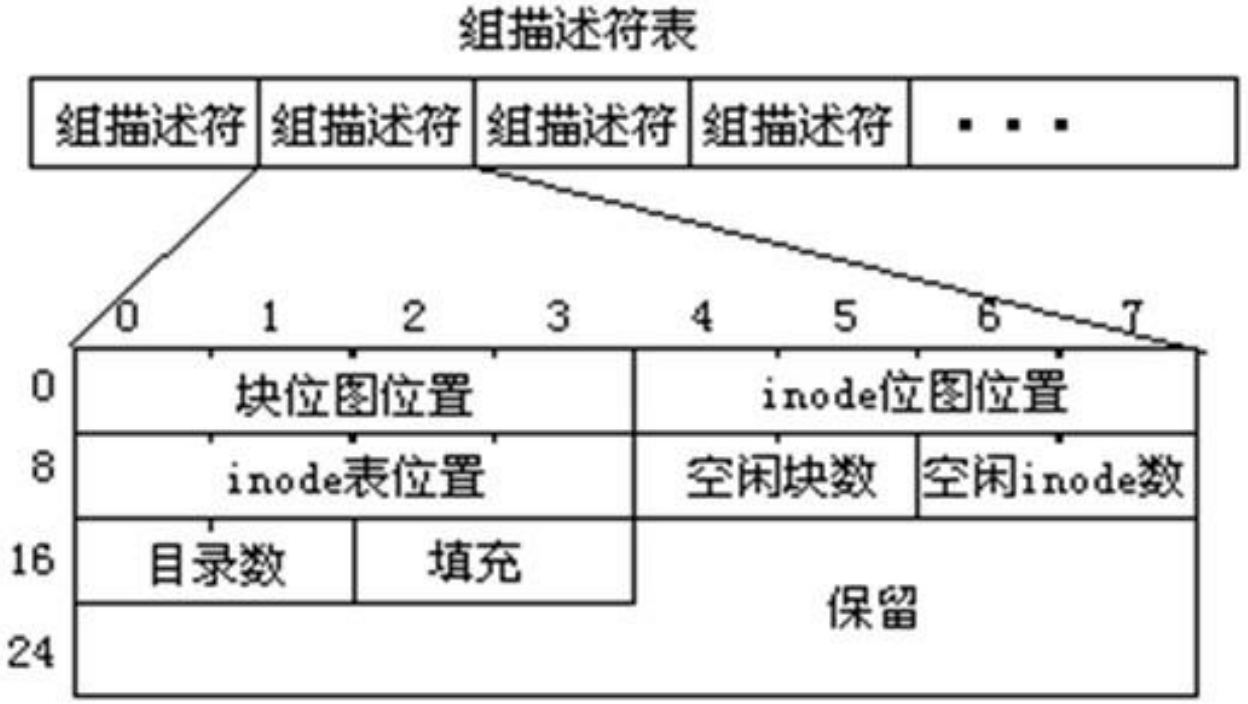


EXT2文件系统超级块

```
struct ext2_super_block {
/* 整个文件系统inode的数量 */
__le32 s_inodes_count;
/* 整个文件系统逻辑块的数量 */
__le32 s_blocks_count;
/* Reserved blocks count */
__le32 s_r_blocks_count;
/* 文件系统剩余逻辑块的数量 */
__le32 s_free_blocks_count;
/* 文件系统剩余inode的数量 */
__le32 s_free_inodes_count;
/* First Data Block */
__le32 s_first_data_block;
/* Block size */
__le32 s_log_block_size;
/* Fragment size */
__le32 s_log_frag_size;
/* 每一个块组中逻辑块的数量 */
__le32 s_blocks_per_group;
...
};
```

块组描述符

- 块组描述符表描述了文件系统中所有块组的属性，存放于超级块所在块的下一个块中。
- 整个分区分成多个块组就对应有多少个块组描述符，每个块组描述符存储一个块组的描述信息。



EXT2文件系统组描述符

```
struct ext2_group_desc
{
__le32 bg_block_bitmap; // 块位图所在的第一个块的块ID
__le32 bg_inode_bitmap; // inode位图所在的第一个块的块ID
__le32 bg_inode_table; // inode表所在的第一个块的块ID
__le16 bg_free_blocks_count; // 块组中未使用的块数
__le16 bg_free_inodes_count; // 块组中未使用的inode数
__le16 bg_used_dirs_count; // 块组分配的目录的inode数
__le16 bg_pad;
__le32 bg_reserved[3];
};
```

数据块位图和索引结点块位图

- EXT2的空闲盘块分配算法采用了位图法
- 位图：为便于查找数据块或索引结点的分配信息
- 每个位（bit）都对应了一个磁盘块：
  - 0，表示对应的磁盘块（或索引结点）空闲
  - 1，表示占用。

4KB 包含 2^12 * 2^3 个位 （一个字节8位）
一个块大小为4096字节

2^12 * 2^3 * 2^12 B = 2^27 B =128 MB

- 2个位图（数据块和索引节点）分别占用一个专门的磁盘块；位于组描述符表之后
- 根据磁盘块的大小，可以计算出每个块组中最多能容纳的数据块个数和索引节点块个数（若磁盘块大小为4KB，则数据块大小为？）

索引结点

- EXT2 中所有的索引结点大小相同，都是 128 个字节。
- 一个inode的结构如图：

```
struct ext2_inode {
__le16 i_mode; // 文件格式和访问权限
__le16 i_uid; // 文件所有者ID的低16位
__le32 i_size; // 文件字节数
__le32 i_atime; // 文件上次被访问的时间
__le32 i_ctime; // 文件创建时间
__le32 i_mtime; // 文件被修改的时间
__le32 i_dtime; // 文件被删除的时间（如果存在则为0）
__le16 i_gid; // 文件所有组ID的低16位
__le16 i_links_count; // 此inode被连接的次数
__le32 i_blocks; // 文件已使用和保留的总块数（以512B为单位）
__le32 i_flags; // 此inode访问数据时ext2的实现方式
union {
struct {
__le32 l_i_reserved1; // 保留
} linux1;
struct {
__le32 h_i_translator; // “翻译者”标签
} hurd1;
struct {
__le32 m_i_reserved1; // 保留
} masix1;
} osd1; // 操作系统相关数据
__le32 i_block[EXT2_N_BLOCKS]; // 定位存储文件的块的数组，前
块号，第14个为二级间接块号，第15个为三级间接块号
__le32 i_generation; // 用于NFS的文件版本
__le32 i_file_acl; // 包含扩展属性的块号，老版本中为0
__le32 i_dir_acl; // 表示文件的“High Size”，老版本中为0
__le32 i_faddr; // 文件最后一个段的地址
union {
struct {
__u8 l_i_frag; // 段号
__u8 l_i_fsize; // 段大小
__u16 i_pad1;
__le16 l_i_uid_high; // 文件所有者ID的高16位
__le16 l_i_gid_high; // 文件所有组ID的高16位
__u32 l_i_reserved2;
} linux2;
struct {
__u8 h_i_frag; // 段号
__u8 h_i_fsize; // 段大小
__le16 h_i_mode_high;
__le16 h_i_uid_high; // 文件所有者ID的高16位
__le16 h_i_gid_high; // 文件所有组ID的高16位
__le32 h_i_author;
} hurd2;
struct {
__u8 m_i_frag; // 段号
__u8 m_i_fsize; // 段大小
__u16 m_pad1;
__u32 m_i_reserved2[2];
} masix2;
} osd2; // 操作系统相关数据
};
```

关于索引节点中的i_block[]

- ext2的索引结点中使用了组合索引方式。

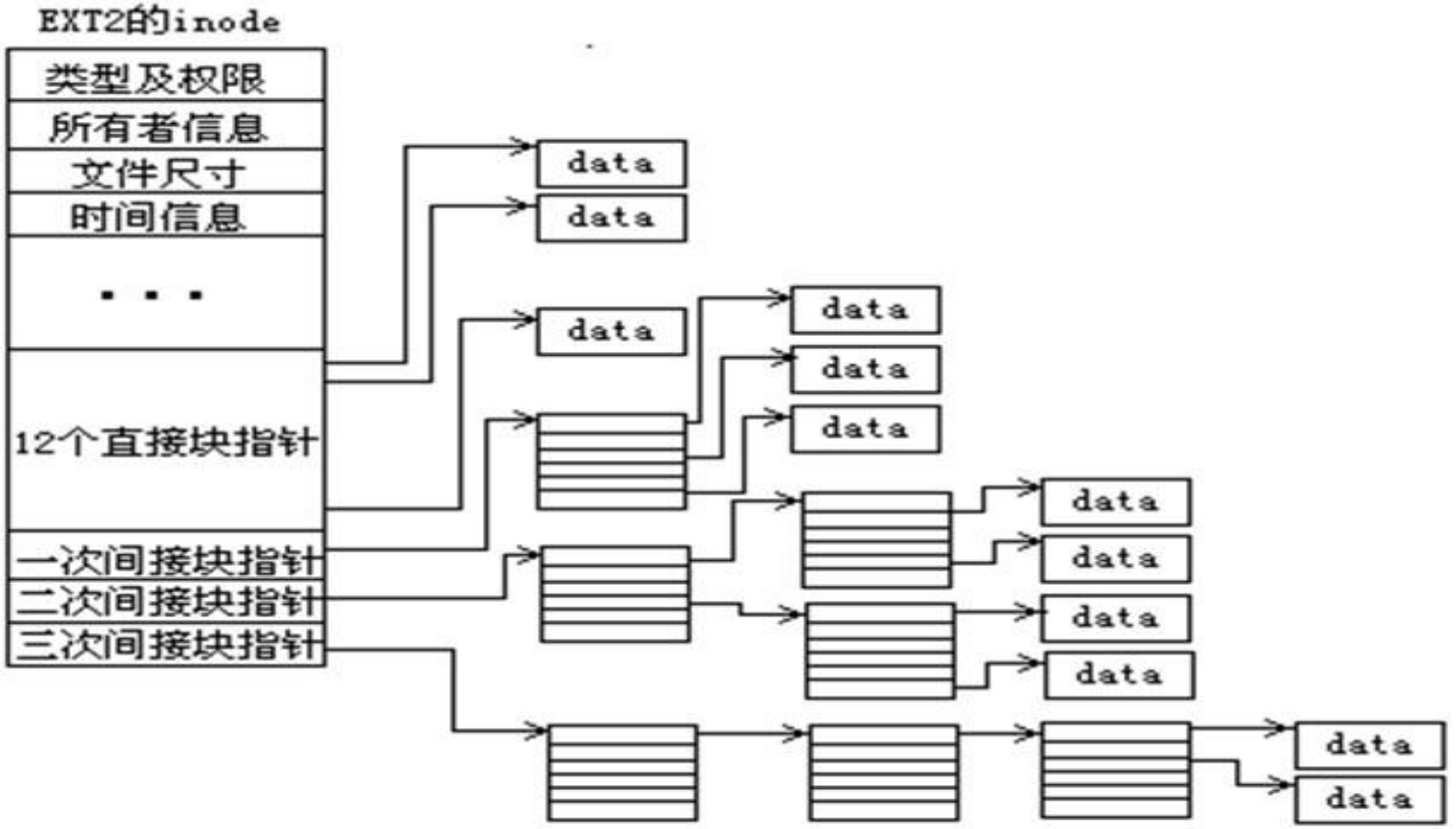
```
00241:      __le32    i_block[EXT2_N_BLOCKS];/* Pointers to blocks */
```

```
00159: /*
00160:  * Constants relative to the data blocks
00161:  */
00162: #define   EXT2_NDIR_BLOCKS          12
00163: #define   EXT2_IND_BLOCK            EXT2_NDIR_BLOCKS
00164: #define   EXT2_DIND_BLOCK           (EXT2_IND_BLOCK + 1)
00165: #define   EXT2_TIND_BLOCK           (EXT2_DIND_BLOCK + 1)
00166: #define   EXT2_N_BLOCKS             (EXT2_TIND_BLOCK + 1)
```

  - 前12项用作直接索引
  - 第13项用作间接索引
  - 第14项用作二次间接索引
  - 第15项用作三次间接索引

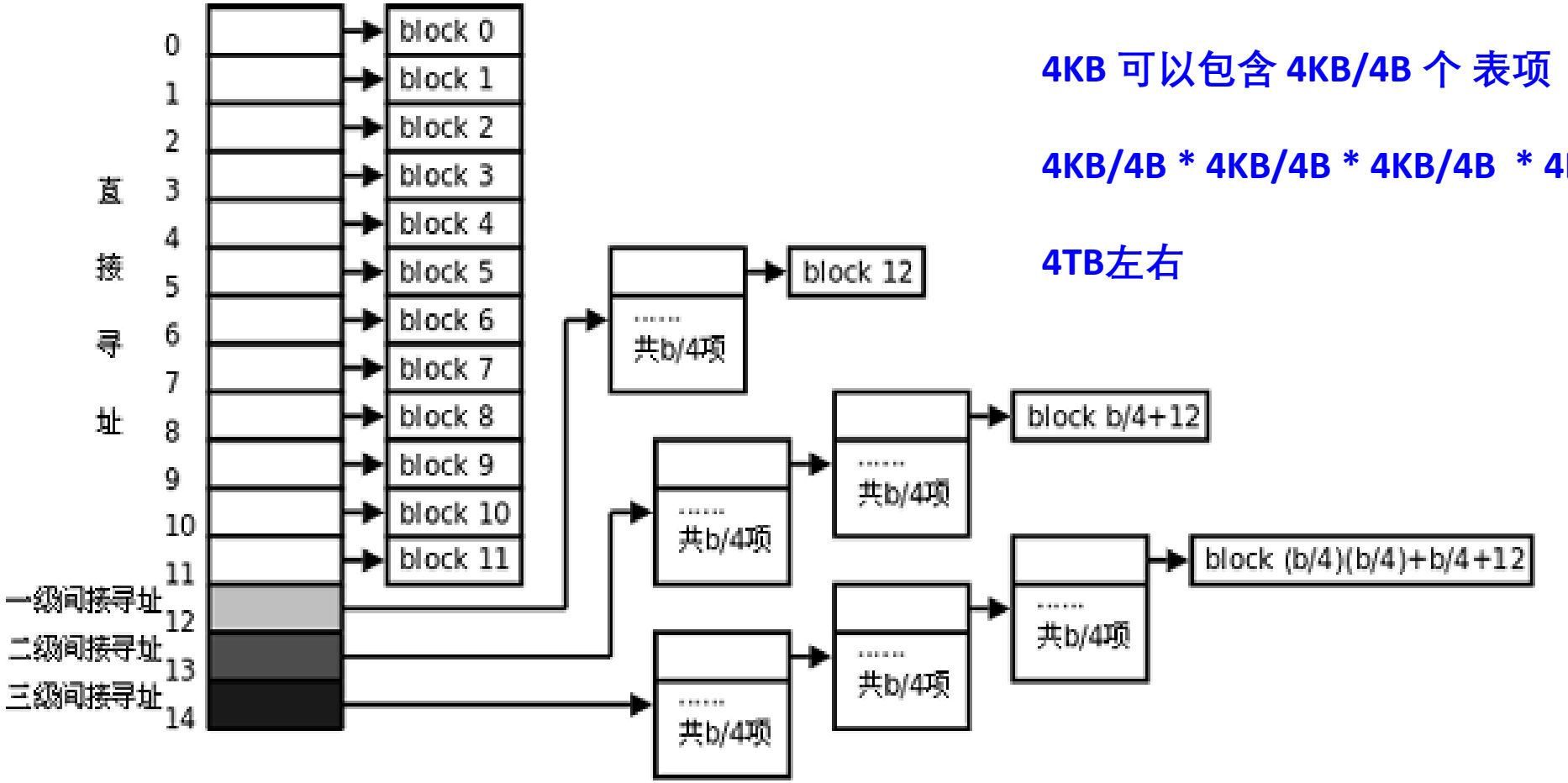
多级索引

- EXT2文件系统中的每个文件由一个inode描述，且只能由一个inode描述。
- inode与文件一起存放在外存，系统运行时，把inode写入内存建立映像，加快文件系统速度。



EXT2的inode中物理块指针示意图

EXT2中的数据块寻址



4KB 可以包含 4KB/4B 个 表项

4KB/4B * 4KB/4B * 4KB/4B * 4KB

4TB左右

- 假如采用磁盘块大小为4KB，索引表项中块号信息占四个字节，那么三级索引能够支持的最大文件多大？

EXT2中的目录项和文件类型

- 在ext2文件系统中，目录是作为文件存储的。
- 这种文件的数据块中存放了该目录下的所有目录项

```
struct ext2_dir_entry_2 {
__le32 inode; // 文件入口的inode号，0表示该项未使用
__le16 rec_len; // 目录项长度
__u8 name_len; // 文件名包含的字符数
__u8 file_type; // 文件类型
char name[255]; // 文件名
};
```

目录项在include/linux/ext2_fs.h文件中定义。

EXT2支持的文件类型

- EXT2在目录项中存放了文件的类型信息。文件类型可以是0～7中的任意一个整数。它们分别代表如下含义：
  - 0：文件类型未知；
  - 1：普通文件类型；
  - 2：目录；
  - 3：字符设备；
  - 4：块设备；
  - 5：有名管道FIFO；
  - 6：套接字；
  - 7：符号链接

```
00539: /*
00540:  * Ext2 directory file types.  Only the low 3 bits are used.  The
00541:  * other bits are reserved for now.
00542:  */
00543: enum {
00544:     EXT2_FT_UNKNOWN,
00545:     EXT2_FT_REG_FILE,
00546:     EXT2_FT_DIR,
00547:     EXT2_FT_CHRDEV,
00548:     EXT2_FT_BLKDEV,
00549:     EXT2_FT_FIFO,
00550:     EXT2_FT_SOCK,
00551:     EXT2_FT_SYMLINK,
00552:     EXT2_FT_MAX
00553: };
```

管理ext2的磁盘空间

- 存储在磁盘上的文件与用户所“看到”的文件有所不同：
  - 用户感觉，文件在逻辑上是连续的
  - 而在磁盘上，存储文件数据的磁盘块可能分散在磁盘各处
- 磁盘碎片整理

随堂习题

- 【考研真题】某XXX文件系统最大容量为16TB，以磁盘块为基本分配单位，磁盘块大小为4 KB。文件控制块(FCB)包含一个1024B的索引表区。请回答下列问题：
  - (1)假设索引表区仅采用直接索引结构，索引表区存放文件占用的磁盘块号。索引表项中块号最少占多少字节？可支持的单个文件最大长度是多少字节？
  - (2)假设索引表区采用如下混合结构：第0～7字节采用＜起始块号，块数＞格式表示文件创建时预分配的连续存储空间，其中起始块号占5B，块数占3B；剩余1016字节采用直接索引结构，一个索引项占8B，则可支持的单个文件最大长度是多少字节？为了使单个文件的长度达到最大，请指出起始块号和块数分别所占字节数的合理值并说明理由。

随堂习题

（1）16TB/4KB = 2^32 块，需要至少32位才能表示最大块号，即块号至少4B。1024B/4B * 4KB = 1MB 大小

（2）3字节为 2^24 个块，2^24×4KB + (1016/8)×4KB

（3）各自占4个字节；块数大于等于四即可（但是出现问题），答案是开放的。

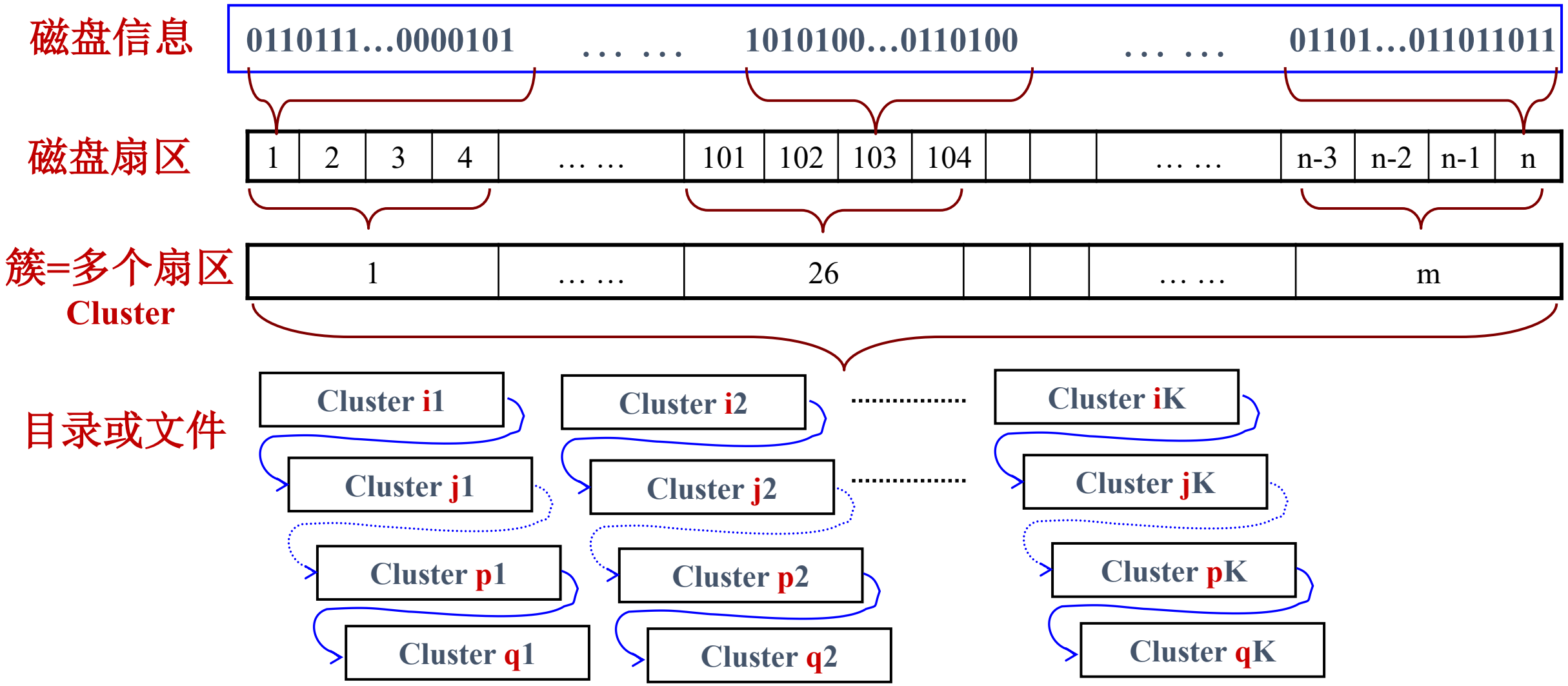
主要内容

- 文件的目录结构
- 文件系统的实现
- EXT2文件系统实现
- Windows的FAT (File Allocation table)文件系统实现

Windows的FAT文件系统实现

数据组织逻辑结构

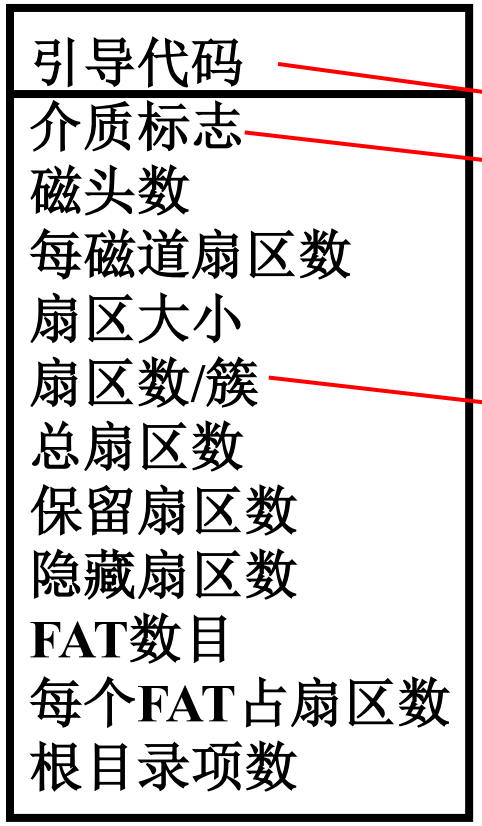
一种特殊的链式结构FS，把所有的链接放到固定长度的FAT表中



Windows的FAT文件系统实现

FAT卷结构示意图





如果是引导分区,则存在该代码,否则空闲

软盘或硬盘等

该参数不是固定不变的, 一般卷越大簇越大

在FAT32中，引导扇区有备份

Windows的FAT文件系统实现

FAT卷结构示意图

引导区	文件分配表 FAT1	文件分配表 FAT2	根目录	其他目录和文件

- 功能：记录和描述整个卷使用情况
- 1. FAT文件系统格式信息：FAT12/FAT16/FAT32
- 2. 卷上每一簇对应FAT中一项，记录该簇使用情况
  - 包含：簇地址号和使用标志信息
  - 使用标志信息＝0 — 该簇空闲未用
  - ≠0 — 该簇被占用
- 3. 每个目录/文件的文件分配链(簇链)
  - 链尾标志信息：0xFFF/0xFFFF/0xFFFFFFFF

FAT表

簇号	链指针
1	3
2	6
3	0xFFF
4	8
5	0xFFF
6	0xFFF
7	0xFFF
8	5
9	0x000

Windows的FAT文件系统实现

FAT卷结构示意图



- FAT2是FAT1的“**镜像**”备份
- 文件分配表对卷非常重要
  - 它的内容破坏会导致部分文件无法访问，甚至导致整卷瘫痪
- 对FAT表备份是十分**必要**的
  - 若FSD（文件系统驱动程序）不能正常访问FAT1，则会访问FAT2

Windows的FAT文件系统实现

FAT卷结构示意图

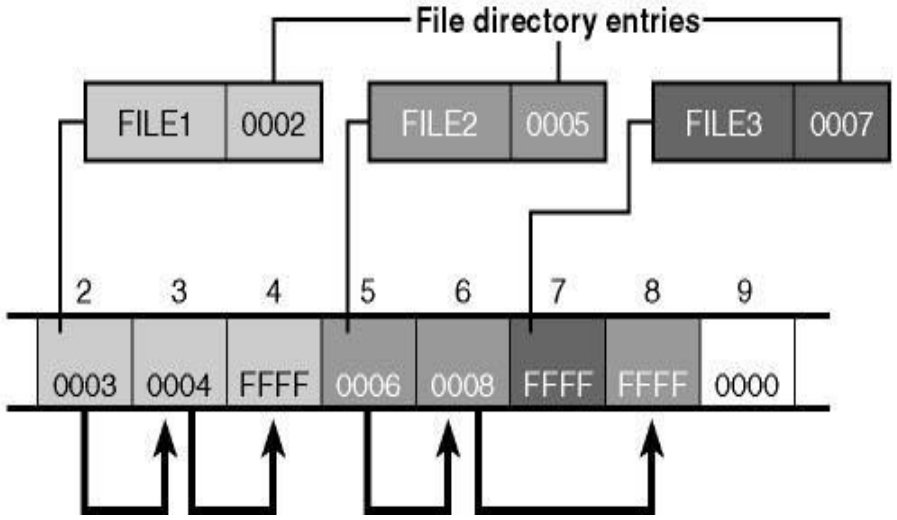


- 根目录区保存卷中根目录项内容
- FAT12和FAT16卷中预留256个目录项空间(是1个文件)
  - 即指定了根目录可以容纳的文件和目录数上限
- FAT32卷中没有预留根目录空间，也无文件/目录数限制
- FAT目录项大小为32字节(文件名遵循8.3命名规则)，保存：文件名、文件尺寸、文件属性、起始簇号、创建日期和时间、最后访问日期、最后修改日期和时间
- 如果目录/文件名为长文件名（非8.3规则），则通过增加若干个目录项的方法解决

FAT卷结构示意图

引导区	文件分配表 FAT1	文件分配表 FAT2	根目录	其他目录和文件

- 其他目录和所有文件存储区
- 该区存放文件的数据内容
- FAT卷的目录同普通文件一样存储的，只是目录文件的内容是子目录和文件项，普通文件内容是一般数据



Windows的FAT文件系统实现

FAT卷结构示意图

引导区	文件分配表 FAT1	文件分配表 FAT2	根目录	其他目录和文件

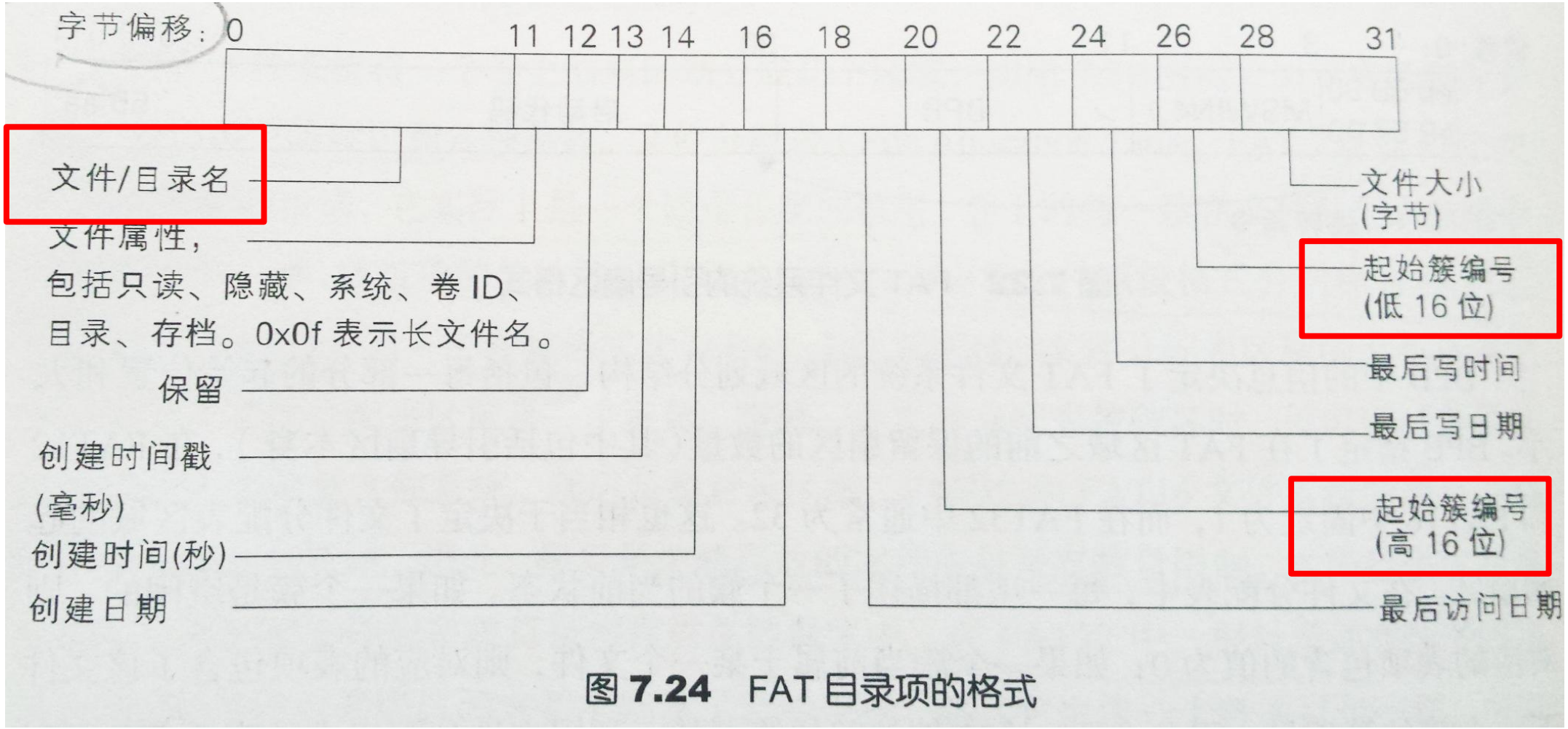


图 7.24　FAT 目录项的格式

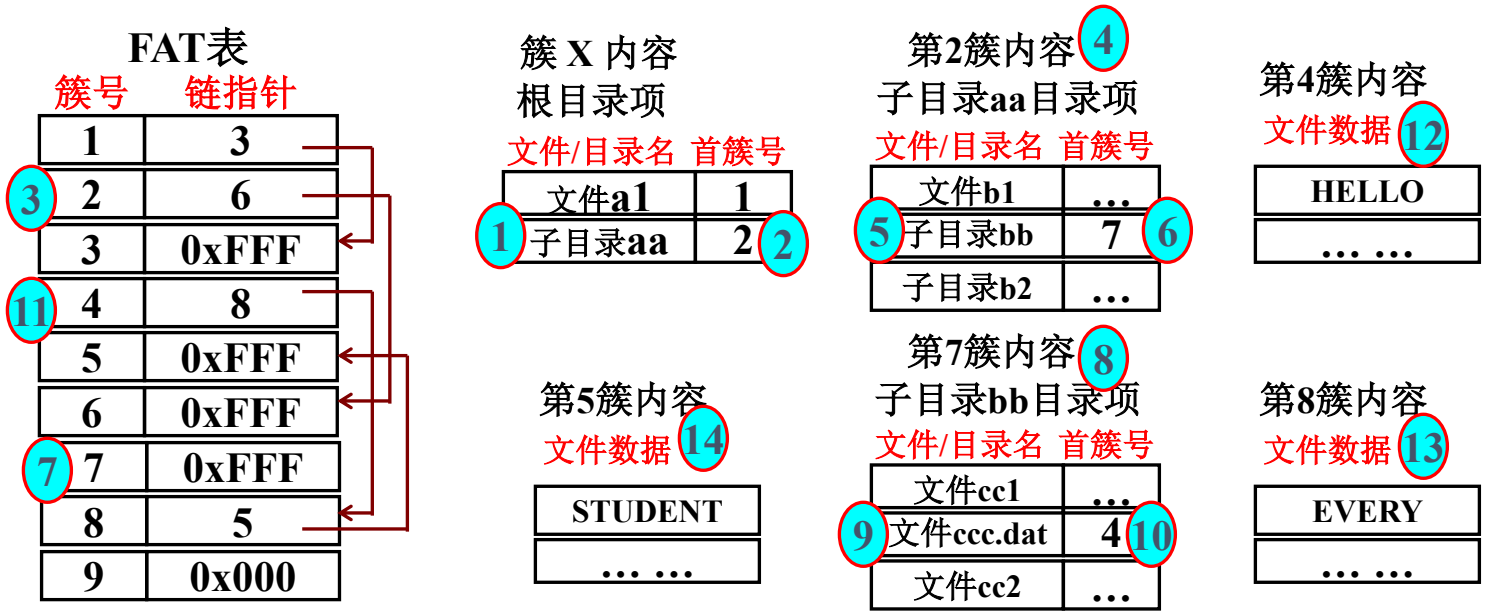
FAT卷结构示意图

引导区	文件分配表 FAT1	文件分配表 FAT2	根目录	其他目录和文件

FAT32长文件名目录项例：The quick brown.fox



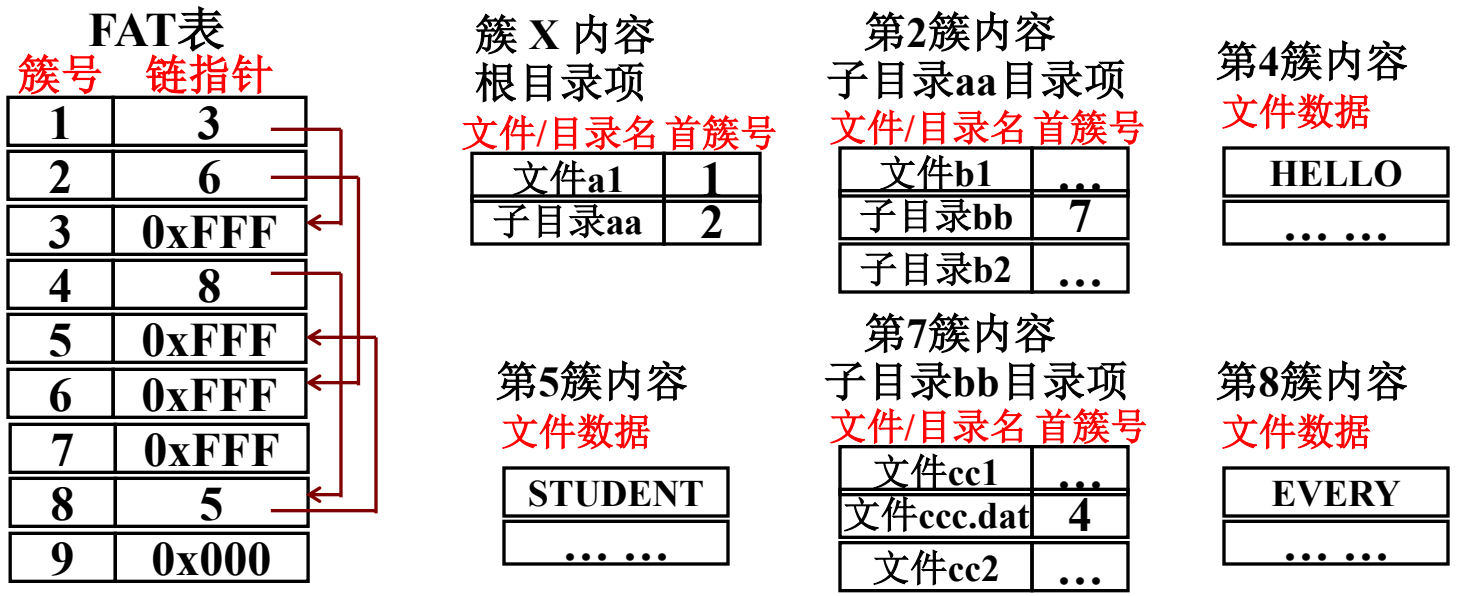
Windows的FAT文件系统实现



文件访问过程：读出文件\aa\bb\ccc.dat的内容

1. 查“根目录”中目录项：找到含目录名=“aa”的目录项
2. 从aa目录项中查出该目录文件的首簇号＝2
3. 查FAT1中以第2簇为头的文件分配簇链，检索相应簇内容找出含目录名=“bb”的目录项
4. 从bb目录项中查出该目录文件的首簇号=7
5. 查FAT1中以第7簇为头的文件分配簇链，检索相应簇内容找出含文件名=“ccc.dat”的目录项
6. 从ccc.dat目录项中查出该文件的首簇号=4
7. 查FAT1中以第4簇为头的文件分配簇链，读出相应簇内容，得到了文件ccc.dat的内容

Windows的FAT文件系统实现



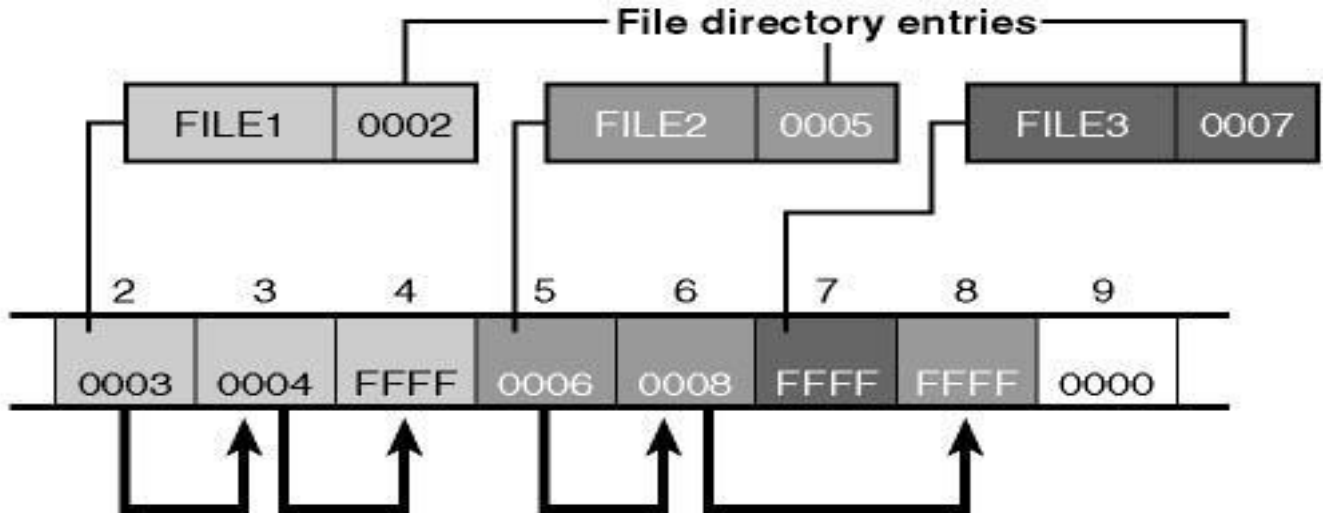
??

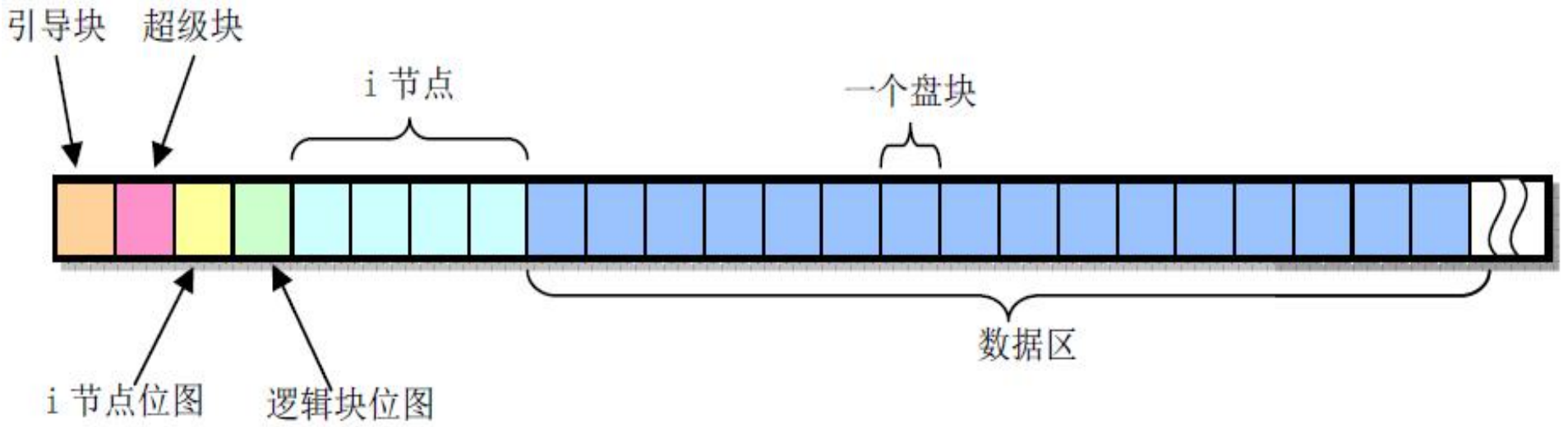
文件删除过程：删除文件\aa\bb\ccc.dat

文件建立过程：建立新文件\aa\fgx.txt

比较两种文件系统

引导区	文件分配表 FAT1	文件分配表 FAT2	根目录	其他目录和文件





Windows的FAT文件系统实现

- FAT卷容量与簇大小的关系

- **FAT12** 最大簇数 = 2^{12}个=4K个
  规定簇大小=0.5KB～8KB
  FAT12卷大小≤32MB
- **FAT16** 最大簇数 = 2^{16}个=64K个
  规定簇大小=0.5KB～64KB
  FAT16卷大小≤4GB
- **FAT32** 最大簇数 = 2^{32}个=4G个
  限定使用2^{28}个=0.25G个
  规定簇大小=4KB～32KB
  FAT32卷大小 ≤ 1TB~8TB

- FAT16/FAT32限制单个文件最大为4GB

W2K系统默认FAT16卷缺省簇大小

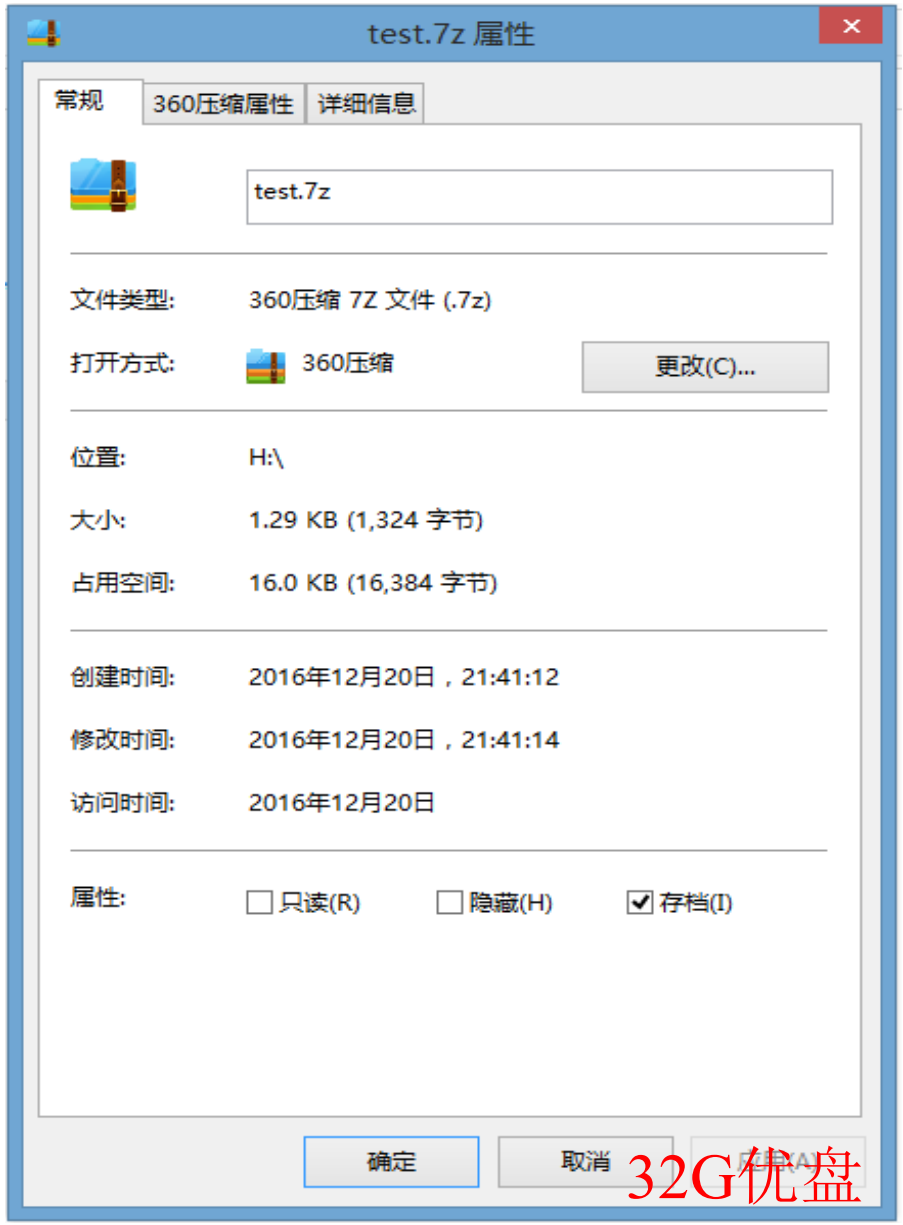
Volume Size	Cluster Size
0-32MB	512bytes
33MB-64MB	1KB
65MB-128MB	2KB
129MB-256MB	4KB
257MB-511MB	8KB
512MB-1023MB	16KB
1024MB-2047MB	32KB
2048MB-4095MB	64KB

FAT32卷缺省簇大小

Volume Size	Cluster Size
32MB-8GB	4KB
8GB-16GB	8KB
16GB-32GB	16KB
>32GB	32KB

为什么在同一种文件系统中簇的大小不统一？ 簇个数固定，但空间大小不定

Windows的FAT文件系统实现



32G优盘



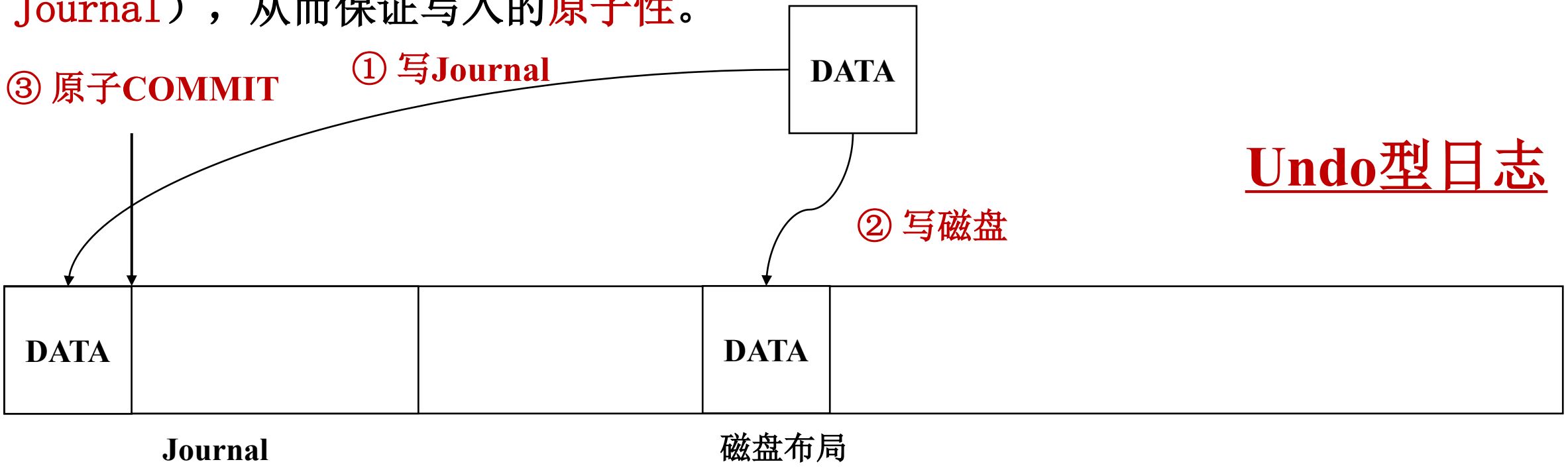
128G优盘

文件系统日志（Journal）

- 在文件系统长期运行的过程中，难免会因为遇到错误使用、电力故障等原因内部外部原因而**发生崩溃**

解决方案：

写数据前先**写到另一个位置**，做个备份。然后将数据一次写到真实位置（**提交Journal**），从而保证写入的**原子性**。

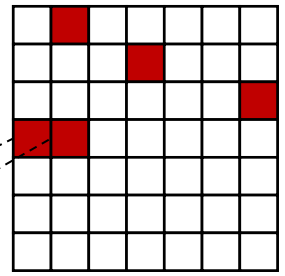


Undo型日志

文件系统日志（Journal）-- 崩溃恢复

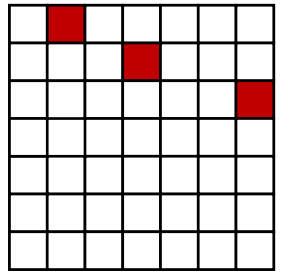
- 基于Undo型Journal的崩溃恢复流程示意

恢复到崩溃前的状态



数据块分配器（位图）
Undo前

③ UNDO



数据块分配器（位图）
Undo后

① 未发现COMMIT标记

② 定位可能不一致的数据

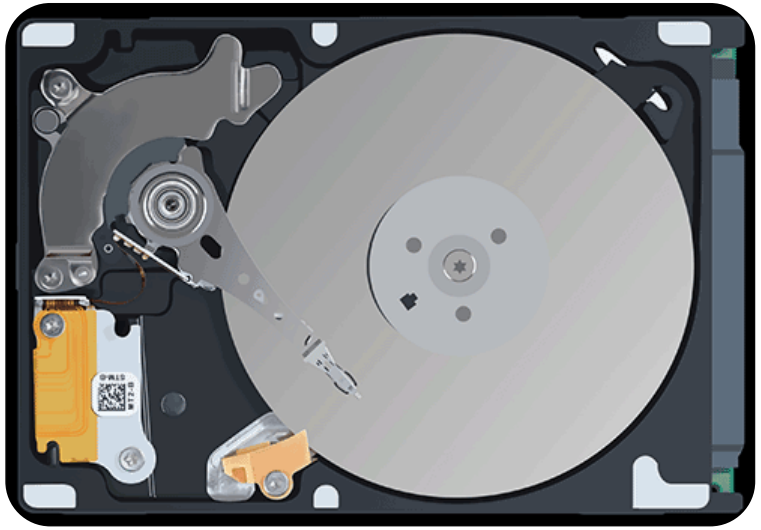


Journal

磁盘布局

日志型文件系统（Log-Structured Filesystem，LFS）

- 传统磁盘文件系统的**随机写入**会带来较高磁盘开销，这是因为磁头、磁盘需要不断通过**机械运动**来寻址。



考虑如下写入序列：

a) Write(file1, 0, ‘a’, 1)

b) Write(file1, 1000, ‘b’, 1)



磁盘布局

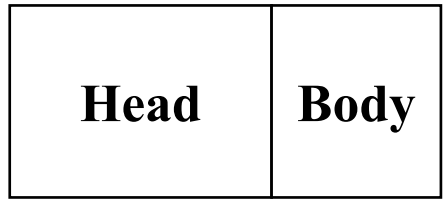
日志型文件系统（Log-Structured Filesystem，LFS）

- 与Journal类似，LFS文件系统**整体**按照**记日记的方式**将数据**顺序**写入存储介质，解决了传统磁盘文件随机写入性能差的问题。

考虑如下写入序列：

a) Write(file1, 0, ‘a’, 1)

b) Write(file1, 1000, ‘b’, 1)



数据实体 / 日志项 / 节点



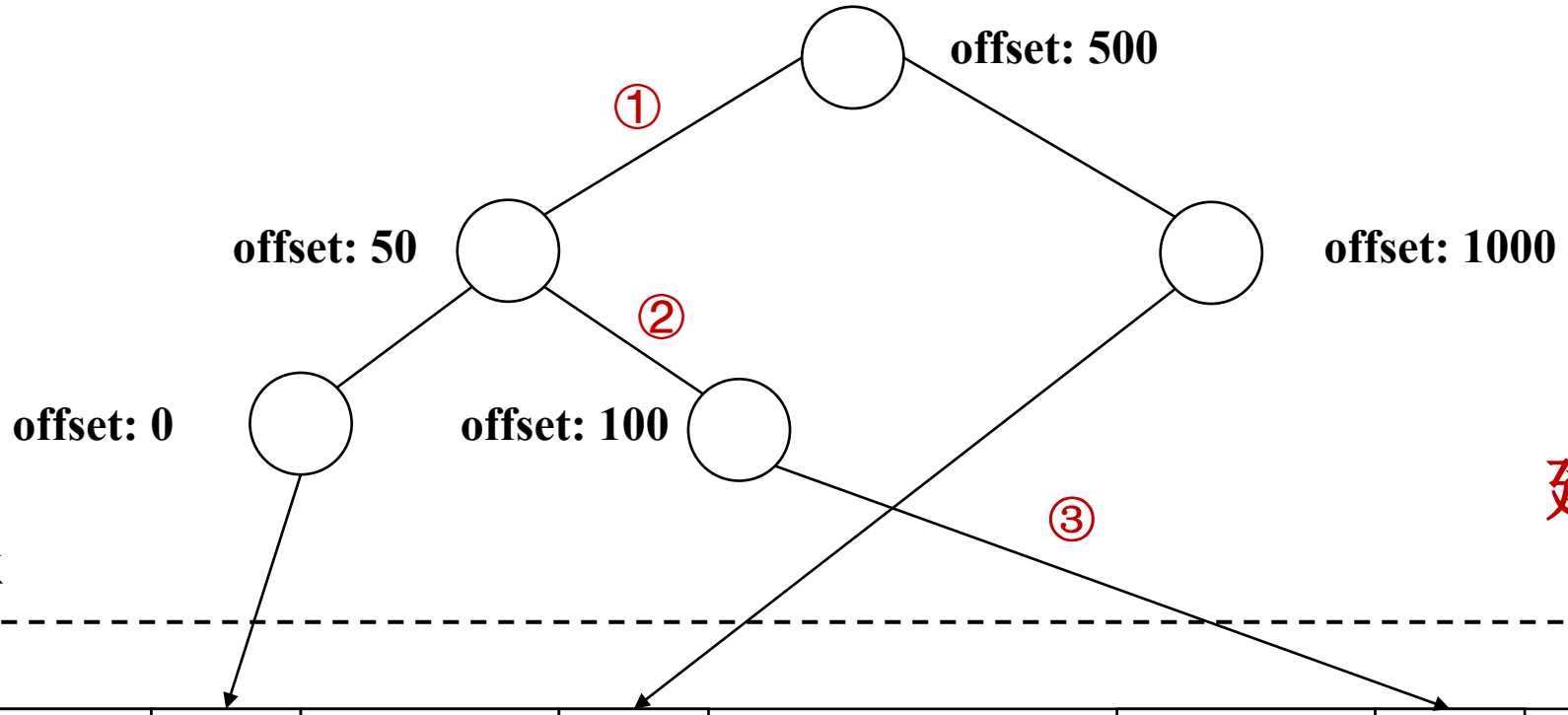
磁盘布局

日志型文件系统（Log-Structured Filesystem，LFS）

- 文件数据以日志方式分布在整个介质上，如何**高效访问数据**？

file1的内存索引:一颗搜索二叉树

read(file1, 100)



建立高效的内存索引!



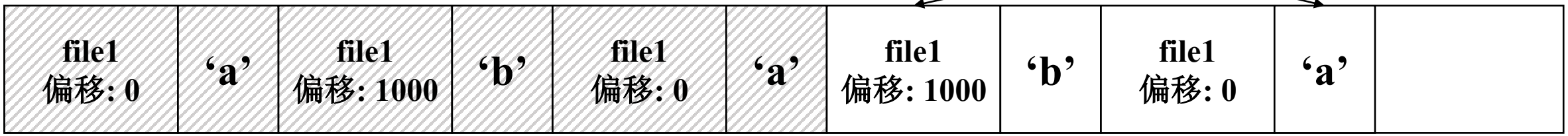
磁盘布局

日志型文件系统（Log-Structured Filesystem，LFS）

- 日志写满了怎么办？一个简单的垃圾回收过程：

①辨别无效数据，收集有效数据

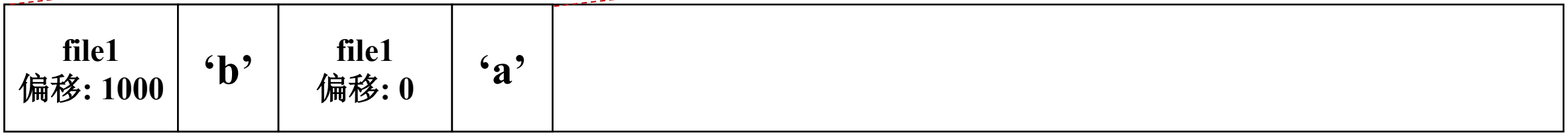
最新写入-有效



②回收无效数据



③搬运有效数据



F2FS（Flash Friendly Filesystem）

- 由于Flash设备具有**磨损寿命**，为了保证磨损均衡，日志型文件系统常常是这类设备的首选；

- F2FS是较新的基于Nand flash的文件系统，其采用了**基于日志追加写**，**数据冷热分离**、**区分顺序写和随机写**的数据布局等技术

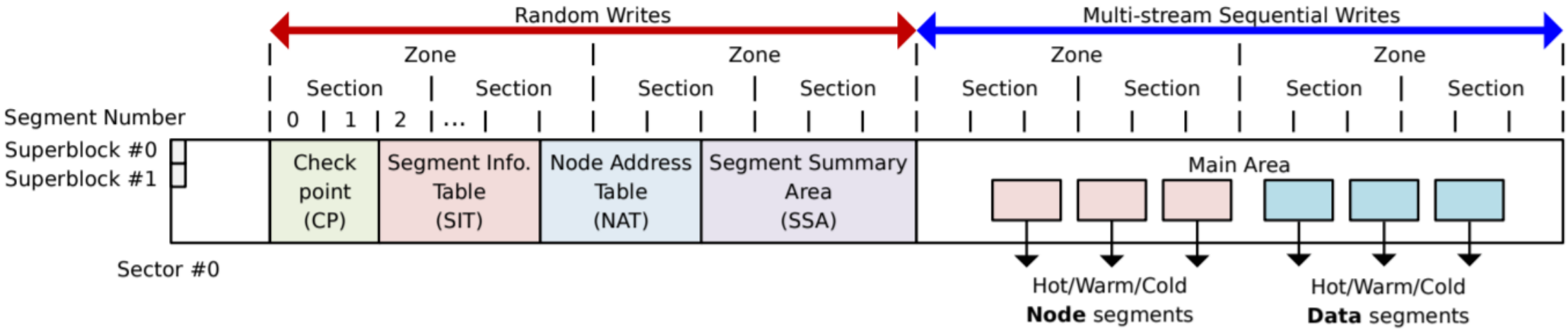
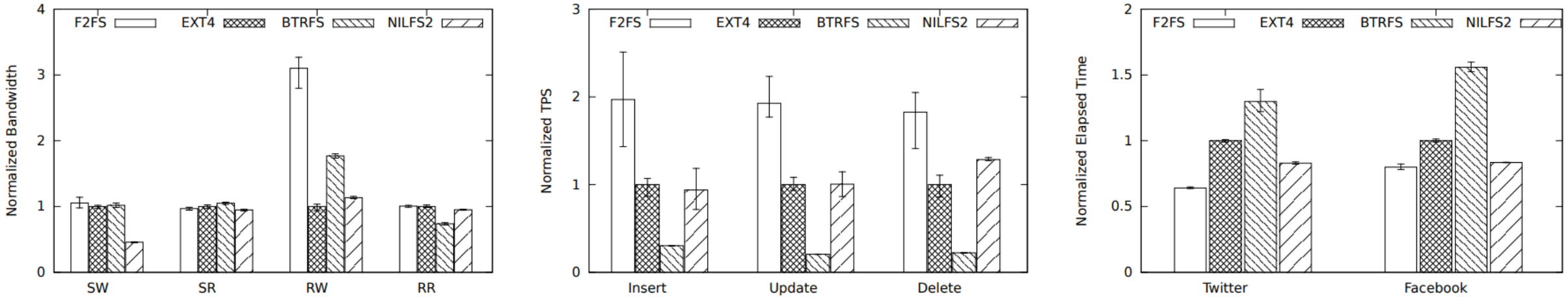


Figure 1: On-disk layout of F2FS.

https://blog.csdn.net/weixin_44465434

F2FS（ Flash Friendly Filesystem ）

- F2FS的提升主要在随机写性能中，比传统EXT4性能好2倍



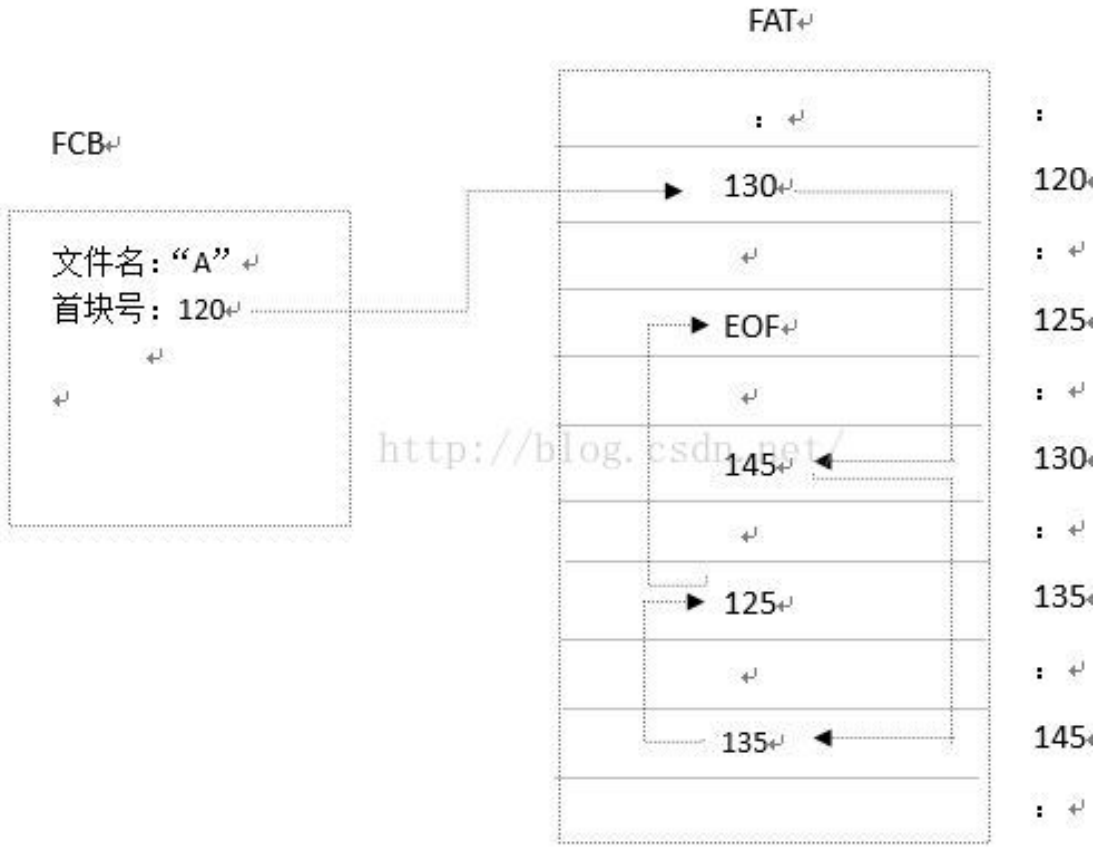
(a) iozone (b) SQLite (c) Facebook-app and Twitter-app

Figure 3: Performance results on the mobile system.

F2FS技术博客：https://blog.csdn.net/weixin_44465434/article/details/113374562

随堂习题

- 假设某文件系统的硬盘空间为500MB，盘块大小为1KB，采用显式链接分配，请回答以下问题：
  - (1)如果约定FAT表项字节数需要与0.5字节对齐，那么其FAT表（文件分配表）需占用多少存储空间？
  - (2)如果文件A占用硬盘的盘块号依次为120、130、145、135、125共五个盘块，请画图示意文件A的FCB与FAT表的关系以及FAT表中各盘块间的链接情况。



例题

- 1、用户在删除某文件的过程中，操作系统不可能执行的操作是（ ）

  A、删除与此文件关联的目录项

  B、删除与此文件对应的文件控制块

  C、释放与此文件关联的内存缓冲区

  D、删除此文件所在的目录

- 答案： D

例题

- 2、若某文件系统索引节点（inode）中有直接地址和间接地址项，则下列选项中，与单个文件长度无关的因素是（ ）

A、索引节点的总数

B、间接地址索引的级数

C、地址项的个数

D、文件块大小

- 答案： A

A. **索引节点的总数**：索引节点的总数反映的是文件系统中文件的总数，而非单个文件的长度。因此，它与单个文件的长度无关。

B. **间接地址索引的级数**：间接地址索引的级数决定了通过多少层间接指针来访问文件的数据块。级数越多，理论上可以访问到的文件长度就越大。因此，它与单个文件的长度有关。

C. **地址项的个数**：无论是直接地址项还是间接地址项，它们的数量都直接影响到可以访问的数据块的数量，从而影响到文件的最大长度。因此，地址项的个数与单个文件的长度有关。

D. **文件块大小**：文件块是文件系统存储数据的基本单位。文件块的大小直接影响到存储相同数据量所需的块数，从而影响到文件的长度。因此，文件块大小与单个文件的长度有关。

例题

- 3、典型文件系统计算题：
  - 1）假定磁盘块的簇大小为1KB，对于540MB的硬盘，其文件分配表（FAT）最少需要占用多少存储空间？
  - 2）某文件系统采用多级索引方式，其inode中有13个地址项，其中直接索引10项，一次间接索引1项，二次间接索引1项，三次间接索引1项。数据块为4KB，磁盘地址用4B表示，这个文件系统允许的最大文件长度是多少？
- 答案：
  - 1）总簇数：540MB/1KB=540K个。因为540K正好小于2^{20}，所以FAT的每个表目占20位，即2.5Bytes，因此FAT的空间为2.5B*540K=1350KB.
  - 2）每个块可以存储的地址数为4KB/4B=1024

    最大文件的块数为$10+1024+1024^2+1024^3$

    故最大文件大小为（$10+1024+1024^2+1024^3$）*4KB = 4TB+4GB+4MB+40KB

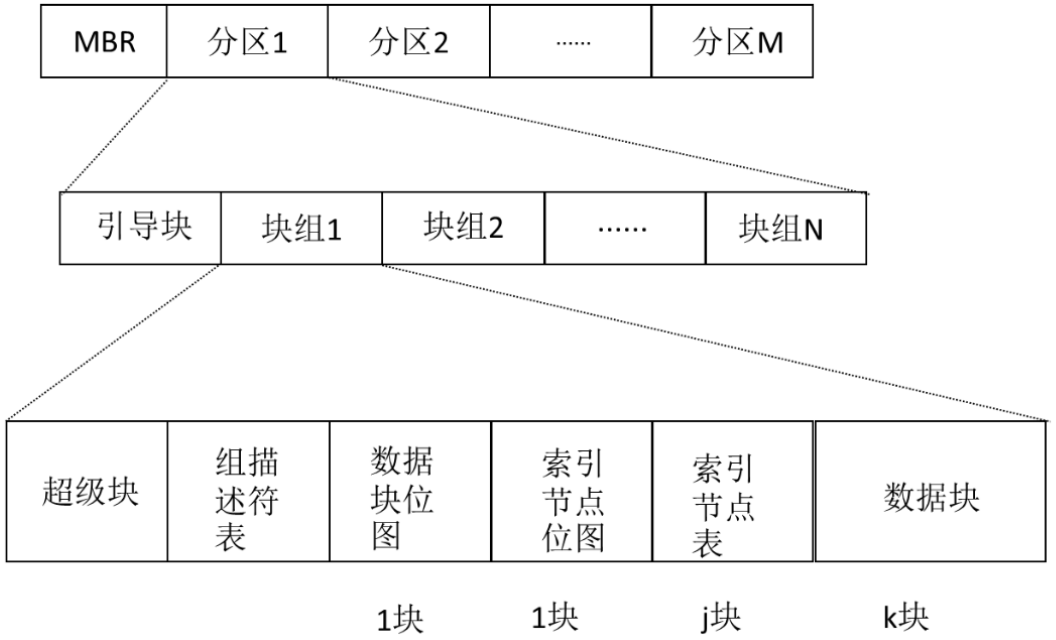
例题

3. Linux 磁盘分区如图所示。除引导块外，每个分区被顺序划分为若干个磁盘块组。Linux 是用位图的方式管理空闲空间，用户在创建一个新文件时，Linux 内核会通过 inode 的位图找到空闲可用的 inode，并进行分配。要存储数据时，会通过块的位图找到空闲的块，并分配。某一个文件系统最大容量为 16TB，以磁盘块为基本分配单位，磁盘逻辑块大小为 4 KB。

（1）若块指针地址为 4 字节，使用 10 个直接地址和一个一次间接块可以访问的最大文件大小是多少字节？（3 分）

（2）若一个索引节点占用 128 字节，那么一个块组中索引节点表和数据块区各占多少个逻辑块？（4 分）

（3）传统磁盘文件系统的随机写入会带来较高磁盘开销，这里磁盘开销主要来源于哪里？你认为可以用什么方式来优化传统磁盘文件系统随机写入慢这个问题？请说明理由。（3 分）



例题

3.

（1）一个间接块中地址指向的磁盘块可以存储 4KB/4B=2^10 个块指针，总共磁盘块有 10+1024=1034 个，文件大小为 1034*4KB=4235264B

（2）索引节点位图占 4KB=2^15bit，因此有 2^15 个索引节点，同理有 2^15 个数据块。索引节点占据的空间是 2^15 × 128B=2^22B=2^12KB，因此需要 2^12KB/4KB=2^10=1024 个逻辑块。

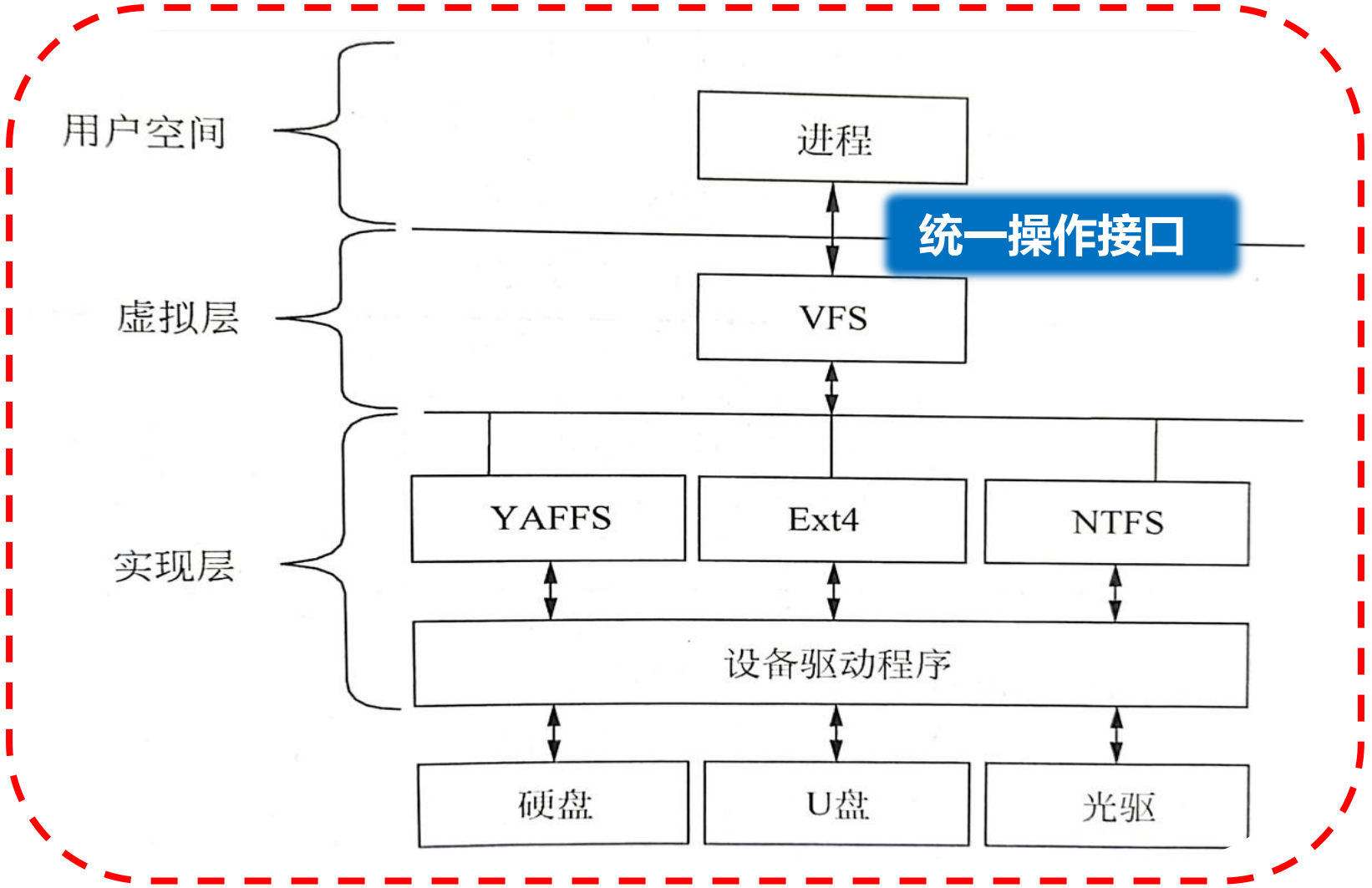
（3）磁盘开销主要来源于磁头、磁盘需要不断通过机械运动来寻址。（1 分）优化方式：① 日志型文件系统，数据追加顺序写入 ② 块缓存 ③ Extent 粗粒度分配结构。④其他言之有理即可。（2 分）

文件系统总结

- 文件系统 ⇒ 帮助用户将信息放在磁盘上
- 一个长的字符序列 ⇒ 盘块集合 ⇒ 文件系统完成映射
- 操作(文件名,偏移) ⇒ 找到文件头(FCB)，找到磁盘盘块
- 系统中会有很多文件 ⇒ 为方便寻找，组织成树状目录
- 目录表明某些文件在一个集合中 ⇒ 目录也是文件，其内容是<文件名，FCB的指针>
- 树状目录 ⇒ 文件路径 ⇒ 路径解析
- 目录+FCB+文件盘块+空闲盘块 = 文件系统
- 从可以运转到良好运转 ⇒ 高效、保护、容错…

openEuler文件系统（扩充知识点）

openEuler 文件系统总体架构：参考《openEuler操作系统》，任炬等

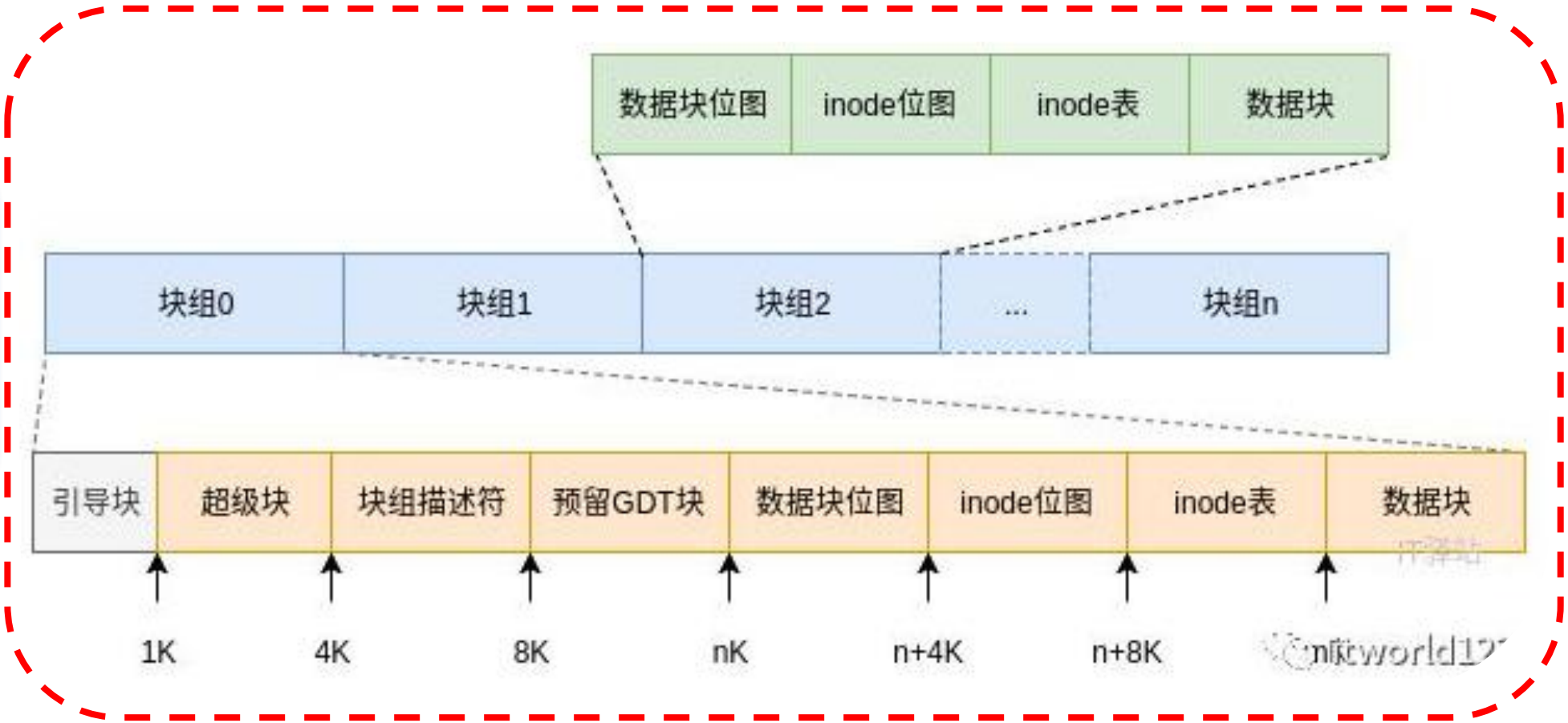


EXT4 基本实现原理： 参考《openEuler操作系统》，任炬等

1. 磁盘整体布局

超级块及块组描述符保存在块组编号为3/5/7及其幂的块组中。

容灾设计!



EXT4 基本实现原理：

参考《openEuler操作系统》，任炬等

2. 超级块

超级块
部分成员

/fs/ext4/ext4.h

```
struct ext4_super_block {
    __le32   s_inodes_count; /* i节点总数 */
    __le32   s_blocks_count_lo;    /* 文件系统总块数 */
    __le32   s_free_blocks_count_lo;/* 空闲块总数 */
    __le32   s_free_inodes_count; /* 空闲i节点总数 */
    __le32   s_first_data_block;    /* 起始数据块号 */
    __le32   s_log_block_size;      /* 块大小 */
    __le32   s_blocks_per_group;  /* 每块组中的盘块数量 */
    __le32   s_inodes_per_group;  /* 每块组中的i节点数量 */
    __le32   s_mtime;          /* 文件系统Mount时间 */
    __le16   s_magic;          /* 文件系统魔术数 */
    __le16   s_state;          /* 文件系统状态*/
    s_last_mounted[64] __nonstring;/* 最后mount目录*/
    … };
```

EXT4 基本实现原理：

参考《openEuler操作系统》，任炬等

3. i 节点

i节点
部分成员

/fs/ext4/ext4.h

```
struct ext4_inode {
        __le16  i_mode;     /* 文件类型及访问权限 */
        __le16  i_uid;                  /* 文件主标识符 */
        __le32  i_size_lo;  /* 文件有多少个字节 */
        __le32  i_size_high; /* 文件有多少个字节 */
        __le32  i_atime;    /* 上一次访问时间 */
        __le32  i_ctime;    /* 上一次 i 节点修改时间 */
        __le32  i_mtime;    /* 上一次文件修改时间 */
        __le32  i_dtime;    /* 文件删除时间 */
        __le16  i_links_count;/* 文件硬链接计数*/
        __le32  i_blocks_lo;/* 文件占多少块*/
        __le32  i_flags;    /* File flags */
... };
```

EXT4 基本实现原理： 参考《openEuler操作系统》，任炬等

5. 块组描述符 /fs/ext4/ext4.h

块组描述符成员（一）

```
struct ext4_group_desc
{
        __le32    bg_block_bitmap_lo;   /* 块位图地址（低32位）*/
        __le32    bg_inode_bitmap_lo;   /*i节点位图地址（低32位）*/
        __le32    bg_inode_table_lo;    /* i节点表起始块号（低32位）*/
        __le16    bg_free_blocks_count_lo; /* 空闲块数量（低16位）*/
        __le16    bg_free_inodes_count_lo; /* 空闲i节点数量（低16位）*/
        __le16    bg_used_dirs_count_lo; /*已经使用的目录项数量（低16位） */
        __le16    bg_flags;                /* 块组标志*/
        __le32  bg_exclude_bitmap_lo;   /* 排除快照的位图*/
        __le16  bg_block_bitmap_csum_lo; /* 数据块位图校验*/
```

EXT4 基本实现原理：

参考《openEuler操作系统》，任炬等

5. 块组描述符

块组描述符成员（二）

/fs/ext4/ext4.h

```
__le16  bg_inode_bitmap_csum_lo;  /* i节点位图校验 */
__le16  bg_itable_unused_lo;  /* 未使用i节点数量*/
__le32  bg_block_bitmap_hi;   /*块位图地址（高32位） */
__le32  bg_inode_bitmap_hi;   /* i节点位图地址（高32位） */
__le32  bg_inode_table_hi;     /* i节点表起始块号（高32位） */
__le16  bg_free_blocks_count_hi; /*空闲块数量（高16位） */
__le16  bg_free_inodes_count_hi;/* 空闲节点数量（高16位） */
__le16  bg_used_dirs_count_hi; ;  /*已经使用的目录项数量（高16位） */
__le16  bg_itable_unused_hi;    /* 未使用节点数量（高16位）*/
__le32  bg_exclude_bitmap_hi;   /*排除快照的位图*/
__le16  bg_block_bitmap_csum_hi;/*排除快照的位图*/
__le16  bg_inode_bitmap_csum_hi i节点位图校验 */
__u32   bg_reserved;
};
```

EXT4 基本实现原理： 参考《openEuler操作系统》，任炬等

6. I/O性能优化—高速缓存

■ 写缓冲技术

- 大部分现代文件系统：推迟5～30S
- 系统调用fsync()可绕过缓冲机制，直接写磁盘

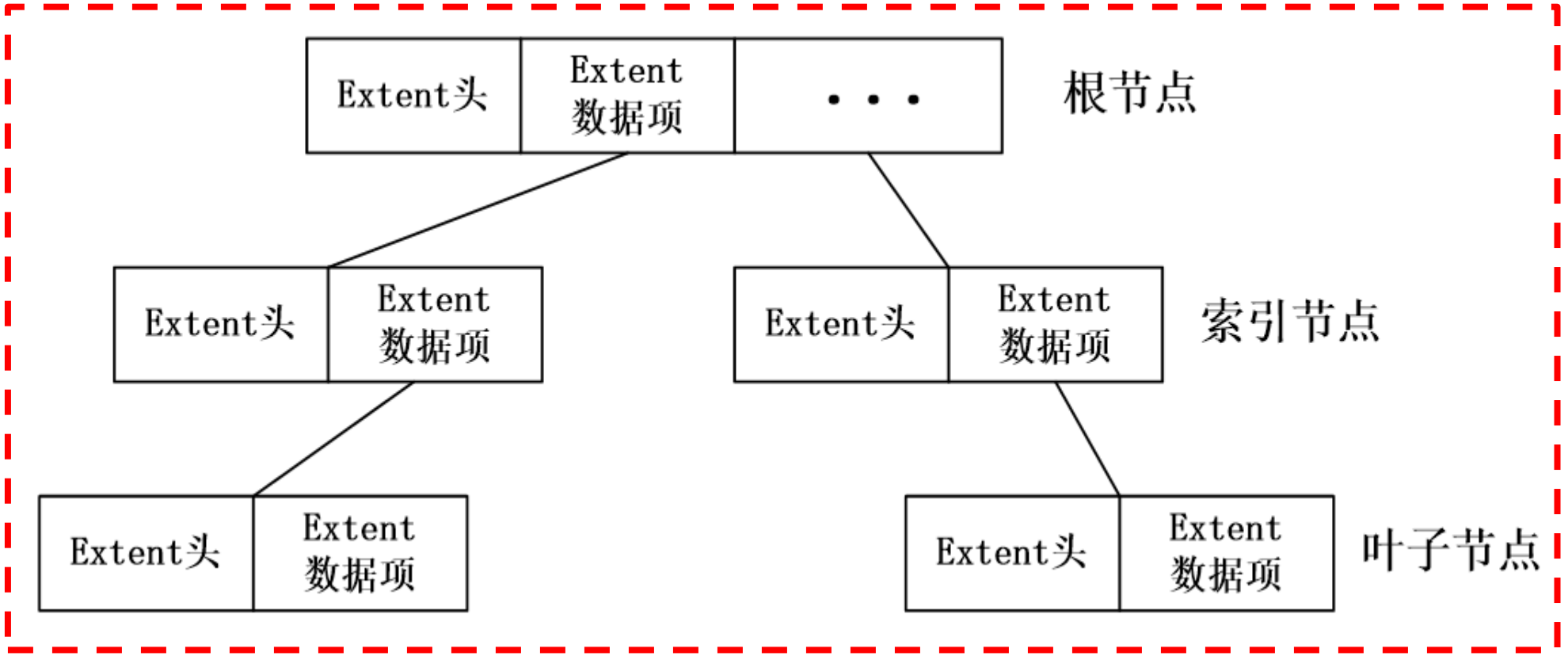
■ 延迟块分配机制：

- 磁盘块分配时间延迟到数据写入磁盘时（页面刷新时）
- 降低文件碎片化程度
- 避免为文件临时内容分配磁盘块

EXT4 基本实现原理： 参考《openEuler操作系统》，任炬等

7. I/O性能优化——Extent

- Extent的B+树结构



EXT4 基本实现原理： 参考《openEuler操作系统》，任炬等

7. I/O性能优化——Extent

- Extent的三种重要数据结构

Extent头

/fs/ext4/
ext4_extents.h

```
struct ext4_extent_header {
        __le16    eh_magic;     /* 用于支持不同的版本*/
        __le16    eh_entries;    /*可用项数量*/
        __le16    eh_max;/* 本区域支持的最大项容量*/
        __le16    eh_depth;     /* 当前树的深度*/
        __le32    eh_generation; /* 第几代的树*/
};
```

EXT4 基本实现原理： 参考《openEuler操作系统》，任炬等

7. I/O性能优化——Extent

- Extent的三种重要数据结构

Extent索引 /fs/ext4/ext4_extents.h

```
struct ext4_extent_idx {
        __le32    ei_block;      /* 下一级Extent节点覆盖的第一个逻辑块块号*/
        __le32    ei_leaf_lo;    /*下一级Extent节点的物理块块号（低32位）*/
        __le16    ei_leaf_hi;    /*下一级Extent节点的物理块块号（高16位） */
        __u16     ei_unused;      /*保留未用字段*/
};
```

EXT4 基本实现原理： 参考《openEuler操作系统》，任炬等

7. I/O性能优化——Extent

- Extent的三种重要数据结构

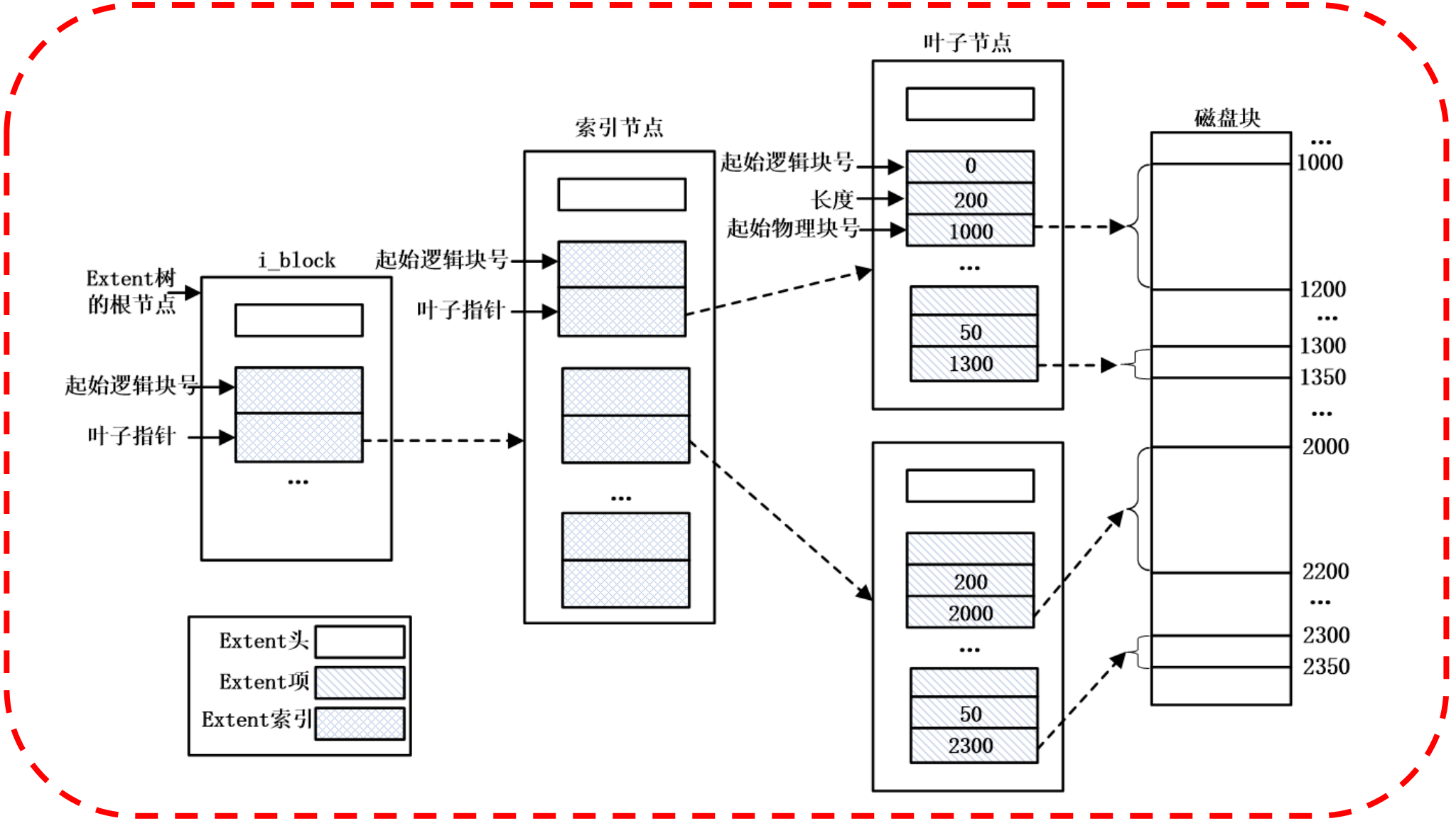
Extent项

/fs/ext4/
ext4_extents.h

```
struct ext4_extent {
        __le32    ee_block;     /* 起始逻辑块号*/
        __le16    ee_len;       /* 连续物理块的块数 */
        __le16    ee_start_hi;  /* 起始物理块号的高16位*/
        __le32    ee_start_lo;  /*起始物理块号的低32位*/
};
```

7. I/O性能优化——Extent

大文件的Extent树结构



Hope you enjoyed the OS course!